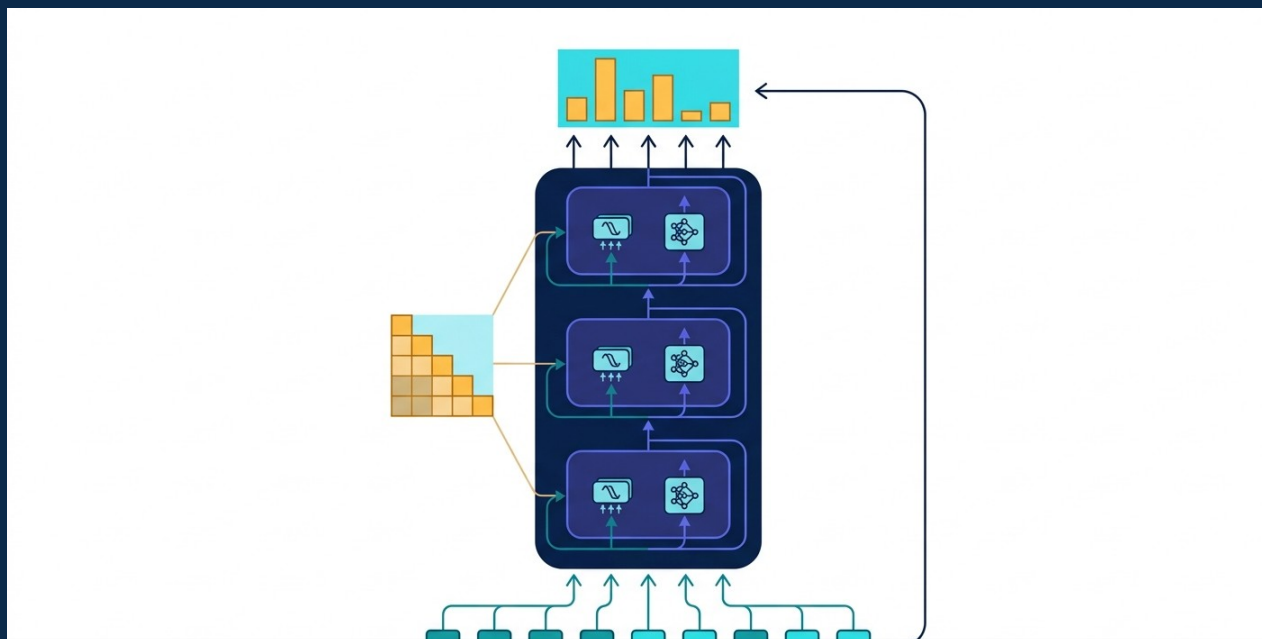




대규모 언어모델과 신약개발

— LLM부터 자율 과학 에이전트까지 · 1교시: LLM 원리 & 과학 언어모델

재직자 3과정 · 6일차(9/5) · 이론 5교시 + 프로젝트(실습) · 강사 박혜진 · 재직자(3년+ 경력)

대규모 언어모델과 신약개발 — 하루 구성 (이론 4H + 프로젝트 4H)

이론 (5교시)

- 1교시: LLM 원리 + 과학 특화 LM
- 2교시: 일반 LLM 활용 + 환각·RAG
- 3교시: 에이전트 원리 + 신약 사례
- 4교시: 하네스 엔지니어링·Agentic 보안
- 5교시: 자율 과학·멀티오믹스·전망

프로젝트 (실습 4H)

- Claude Code 과학 에이전트 직접 구축
- 도구 등록(function calling):
PubMed·ChEMBL·RDKit·ADMET
- 자연어→타겟조사→후보→ADMET→보고서 자율실행
- + 화학·단백질 LM·RAG hands-on · PDE5 하네스

1교시에서 볼 것 — 일반 LLM 원리 → 과학 LM

A. 일반 LLM 원리 (박혜진 교안 도식)

RNN→LSTM→seq2seq→Attention→Q/K/V→Self/Multihead→Transformer→GPT/BERT/T5→RLHF→LLaMA

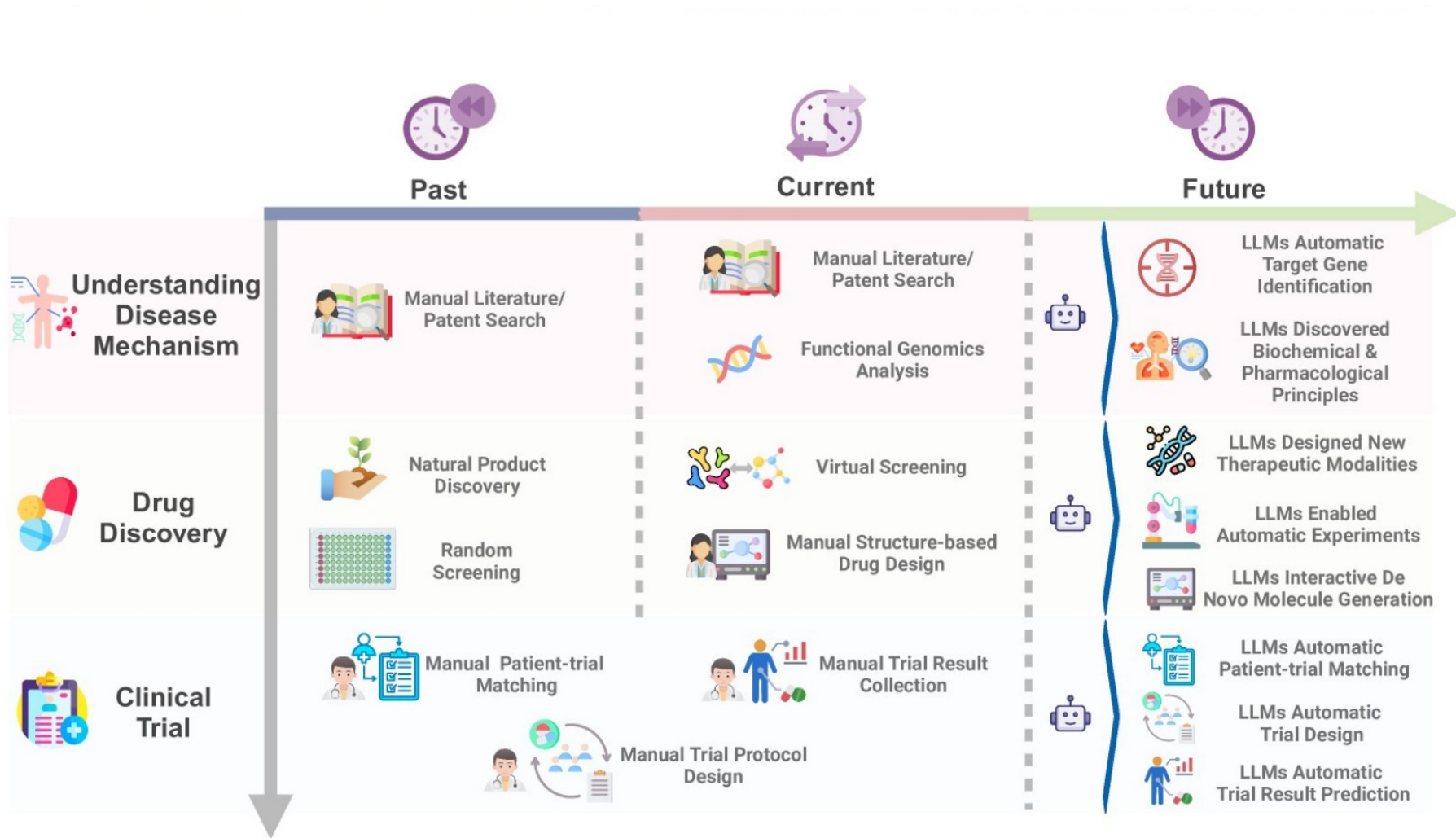
B. 과학 언어모델 (2026 현행)

생물학 LM(ESM3·ESMC·ESMFold2) · 화학 LM(MolT5·ChemDFM·ChemVLM) · 치료제/오믹스 (TxGemma·scGPT)

C. 정리

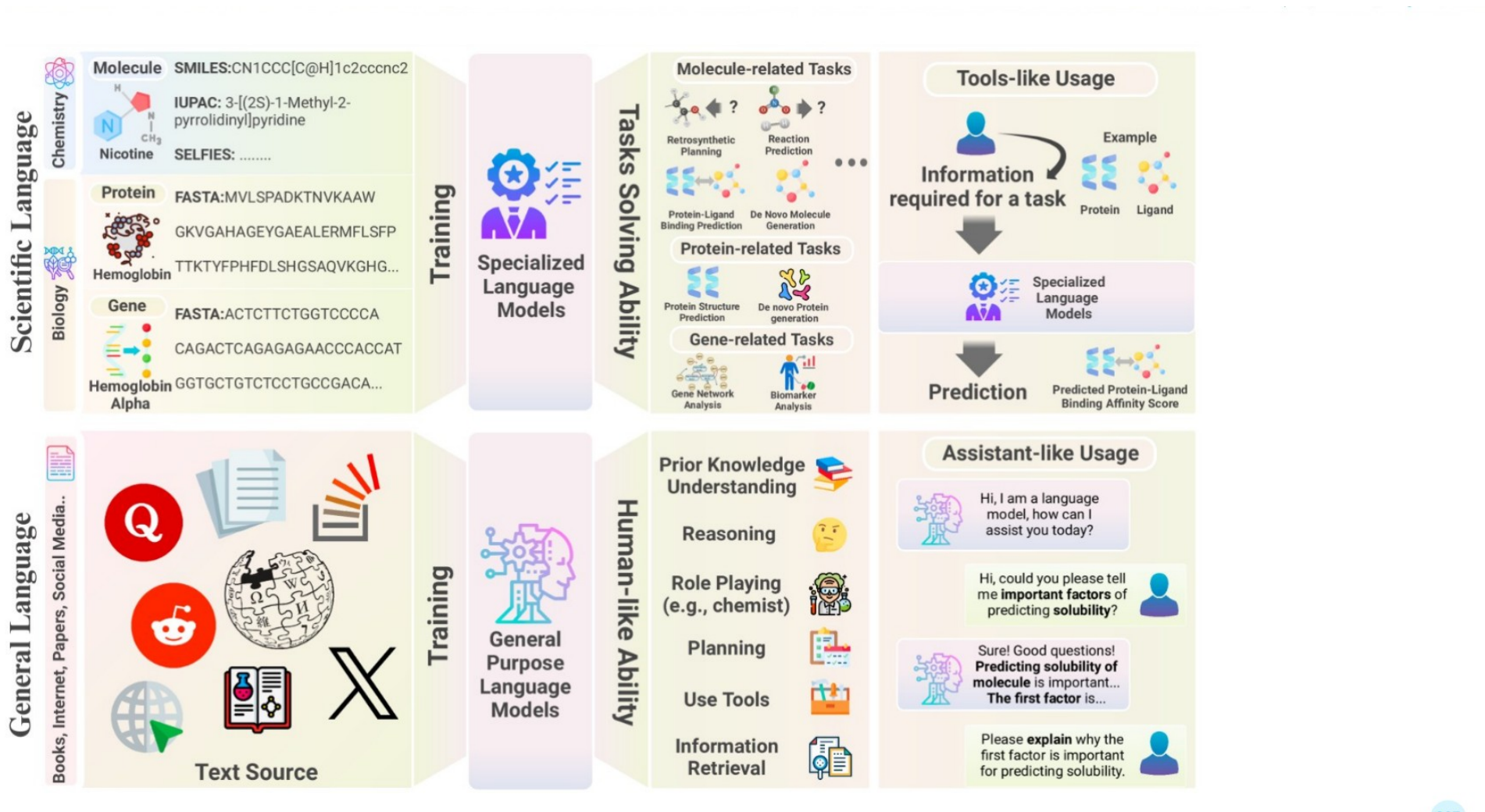
일반 vs 과학 LM 대비 · 2026 지형 · 2교시(활용·환각통제) 예고

왜 신약개발에 LLM인가 — 과거·현재·미래



▶ 신약 지식의 대부분은 비정형 텍스트(논문·특허·실험노트). LLM은 이를 읽고 요약·추출·가설 생성 → 분자·단백질도 '언어'로(PART B)

LLM이 신약개발 전주기에 들어오는 지점



▶ Target ID → Hit/Lead → 최적화 → 전임상 → 임상: 각 단계에 LLM·에이전트가 접목. 오늘은 그 '언어모델' 자체의 원리부터.

PART A

일반 LLM 원리

다음 토큰 예측 → RNN → Attention → Transformer → GPT/BERT → 정렬(RLHF)

언어모델 = 다음 문자/토큰 예측 (char-LM 예시)

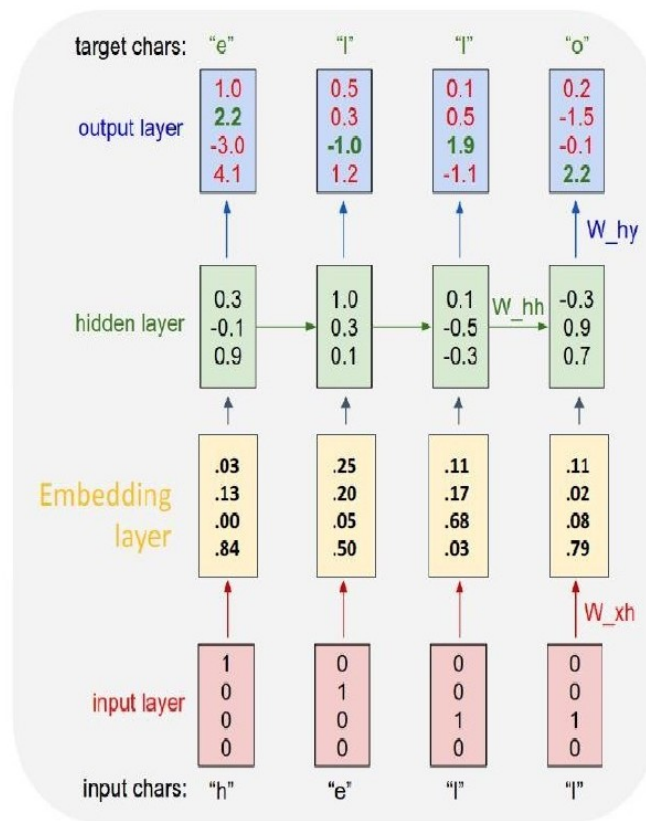
Example: Language Modeling

So far: encode inputs
as **one-hot-vector**

$$\begin{bmatrix} w_{11} & w_{12} & w_{13} & w_{14} \\ w_{21} & w_{22} & w_{23} & w_{24} \\ w_{31} & w_{32} & w_{33} & w_{34} \end{bmatrix} \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} w_{11} \\ w_{21} \\ w_{31} \end{bmatrix}$$

Matrix multiply with a one-hot vector just
extracts a column from the weight matrix.
Often extract this into a separate
embedding layer

- 별도의 임베딩층을 만들 수도 있다.



▶ 'hello'를 한 글자씩: 지금까지 입력으로 다음 글자를 확률 예측 → 샘플링 → 다시 입력. 이 '자기회귀'가 GPT의 뿌리.

언어모델의 수식 — 자기회귀 밀도모델

autoregressive LM

$p(x) = \prod_i p(x_i \mid x_1 \dots x_{i-1})$ # chain rule (다음 토큰 예측)
학습: $L = -\sum_i \log p(x_i \mid x_{<i})$ # 자가지도(라벨 불필요)
생성: start → 샘플링 → 피드백 → 반복 # 한 토큰씩

▶ 문장 확률 = 각 토큰 확률의 곱. '다음 단어 맞히기'를 끝까지 이어붙인 것 — 요약·추론·번역도 이 목표의 부산물

쉽게

언어모델 = '지금까지의 말'을 보고 '다음 단어'를 확률로 맞히는 것. 방대한 문장으로 자가지도 학습 → 맞힌 단어를 다시 입력에 붙여 반복하면 글이 생성된다.

Vanilla RNN의 역전파 - 그래디언트 소멸 문제



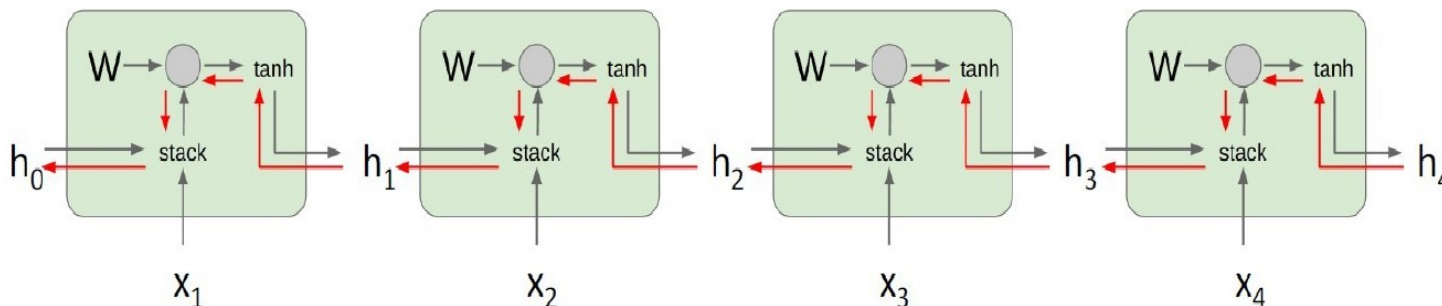
그래디언트 소멸 문제

- 그래디언트는 과거의 미래에 대한 영향이다. 만약 기간가의 거리가 멀어 (예를 들면, 타임스텝 t 과 타임스텝 $t+n$ 의 거리) 그래디언트가 매우 작으면 타임스텝 t 와 $t+n$ 사이의 상관관계가 없는 것인지, 타임스텝 t 와 $t+n$ 간의 참 파라미터를 잘못 이해하는 것인지 구별할 수 없다.

▶ 같은 가중치 W_h 를 시점마다 반복 곱 $\rightarrow$ 긴 문장에서 그래디언트가 0에 수렴(소멸) 또는 발산(폭발). 앞부분을 '잊는다'.

vanishing / exploding → 구조를 바꾸자

Vanilla RNN Gradient Flow



Computing gradient of h_0 involves many factors of W (and repeated $\tanh$)

Largest singular value > 1 :
Exploding gradients

Largest singular value < 1 :
Vanishing gradients

→ **Change RNN architecture!**

● 변화 필요

구조

▶ 폭발은 gradient clipping으로, 소멸은 '구조 변경'으로 해결 → LSTM 등장.

LSTM — 게이트가 있는 기억 셀

LSTM의 도입

Vanilla RNN

$$h_t = \tanh \left(W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix} \right)$$

LSTM

$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$

Two vectors at each timestep:

Cell state

Hidden state

$$c_t = f \odot c_{t-1} + i \odot g$$

$$h_t = o \odot \tanh(c_t)$$

● 매 타임스텝에 2개의 벡터:
셀상태(c)와 은닉상태(h)

▶ cell state(기억 고속도로) + forget·input·output 게이트(0~1 수도꼭지). 덧셈적 갱신으로 장기 의존성 보존.

LSTM 게이트 상세 — 무엇을 지우고·저장하고·내보낼까

- 입력게이트: 셀 쓰기
- 망각게이트: 셀 망각
- 출력게이트: 얼마나 h에 전달
- 게이트게이트: 얼마나 셀에 쓸지

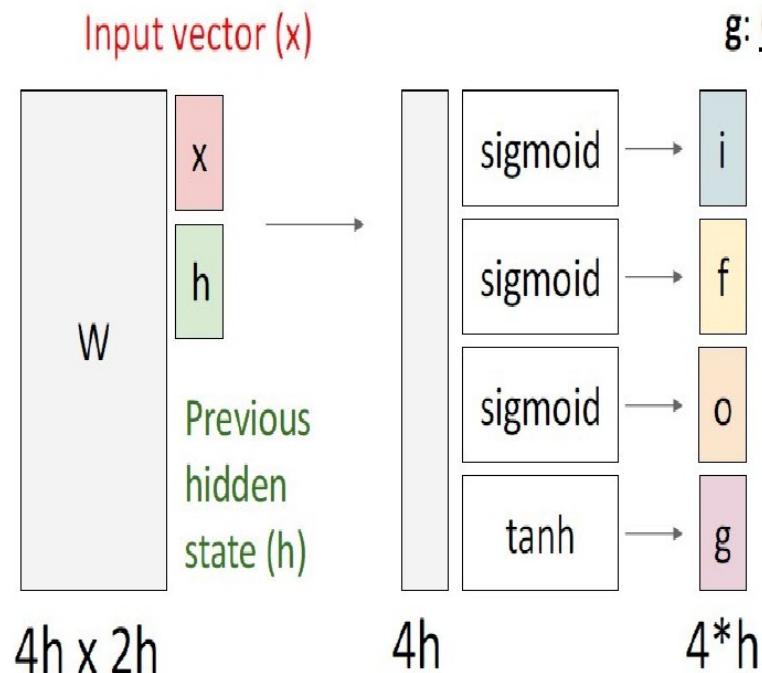
i: Input gate, whether to write to cell

f: Forget gate, Whether to erase cell

o: Output gate, How much to reveal cell

g: Gate gate (?), How much to write to cell

LSTM의 도입



$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$

$$c_t = f \odot c_{t-1} + i \odot g$$

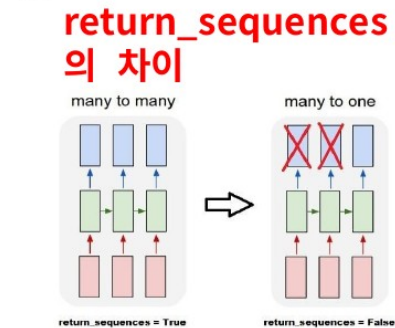
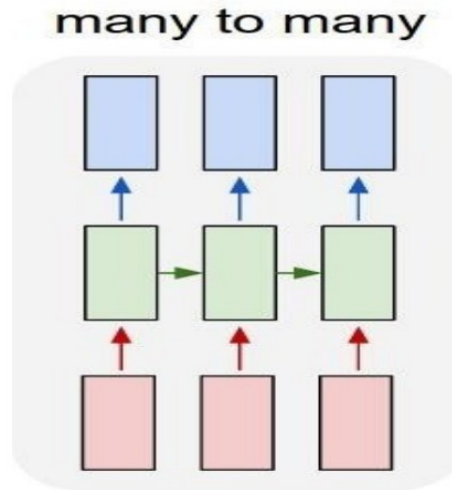
$$h_t = o \odot \tanh(c_t)$$

▶ 매 시점 4개 게이트 계산. GRU(Cho 2014)는 게이트 2개로 단순화. 그래도 '순차 처리'라 병렬 불가 → 느림.

시퀀스 처리 패턴 — one/many-to-one/many

그림

Many to Many



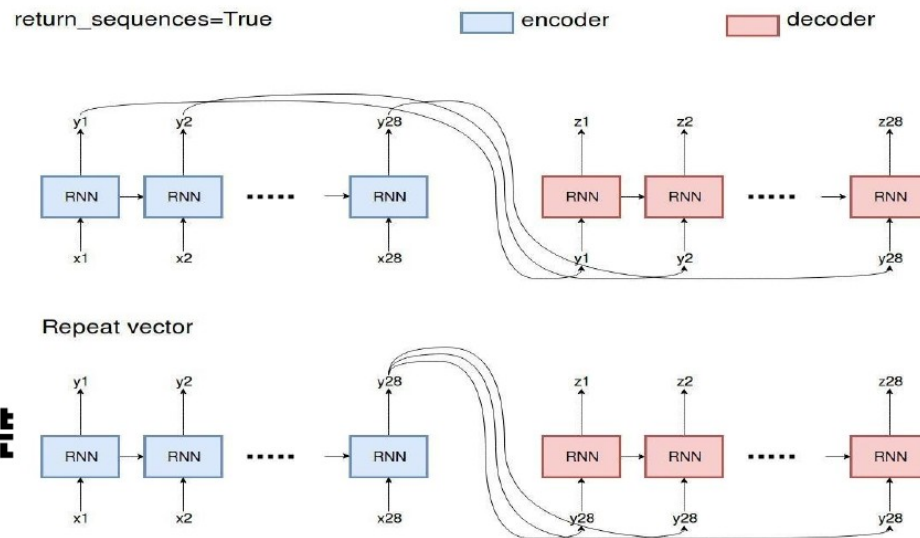
코드

```
model = Sequential()
model.add(LSTM(1, input_shape=(timesteps, data_dim),
return_sequences=True))
```

▷ 입력 · 출력 길이 조합(감성분류=many-to-one, 번역=many-to-many). 번역 · 요약을 위한 Encoder-Decoder로 이어진다.

Encoder-Decoder — 문맥벡터로 압축

그림



Encoder Decoder 기법

코드

```

model.add(LSTM(N_BLOCKS, input_shape=(N_INPUTS,
N_FEATURES))) model.add(RepeatVector(N_OUTPUTS))
model.add(LSTM(N_BLOCKS,
return_sequences=True))
model.add(TimeDistributed(Dense(1)))
model.add(Activation('linear'))

```

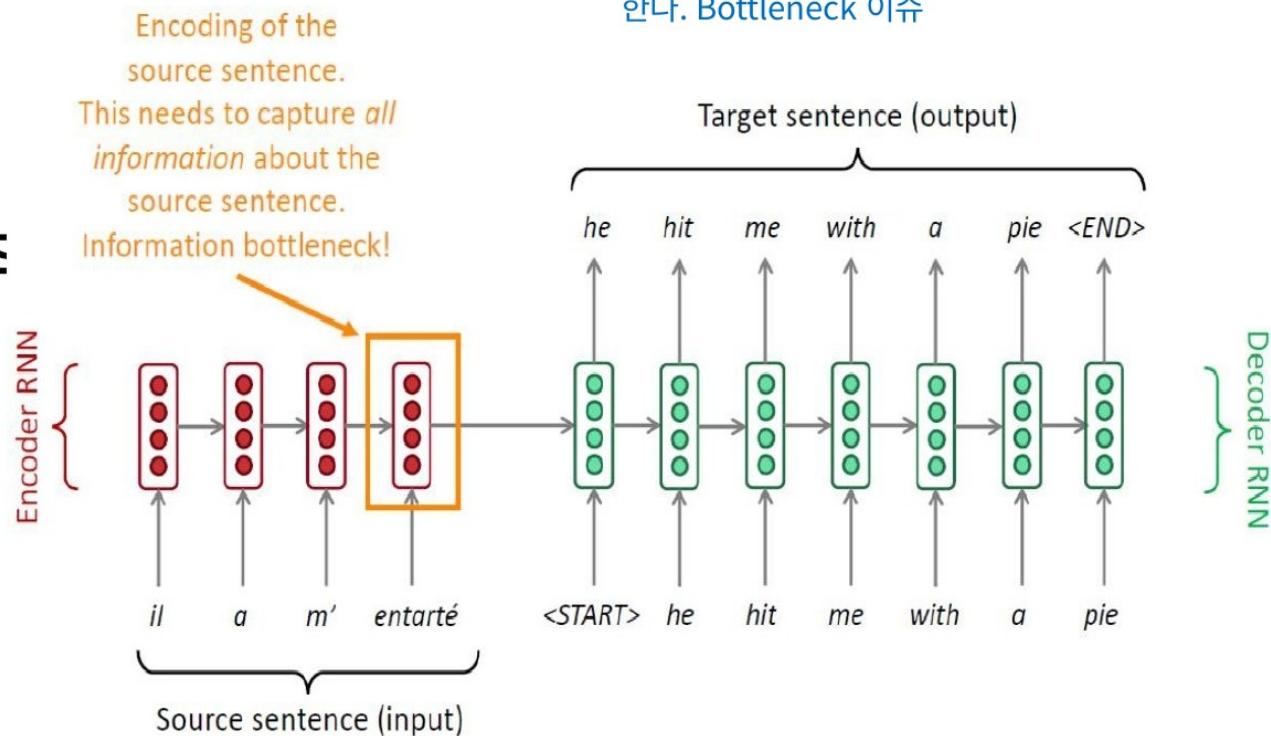
▷ 인코더가 입력을 하나의 벡터로 압축→디코더가 출력 생성. 문제: 긴 문장을 벡터 하나에 다 담는 '병목' → Attention.

Attention 기초 — seq2seq 병목을 푼다

● seq2seq 모델: Bottleneck 이슈

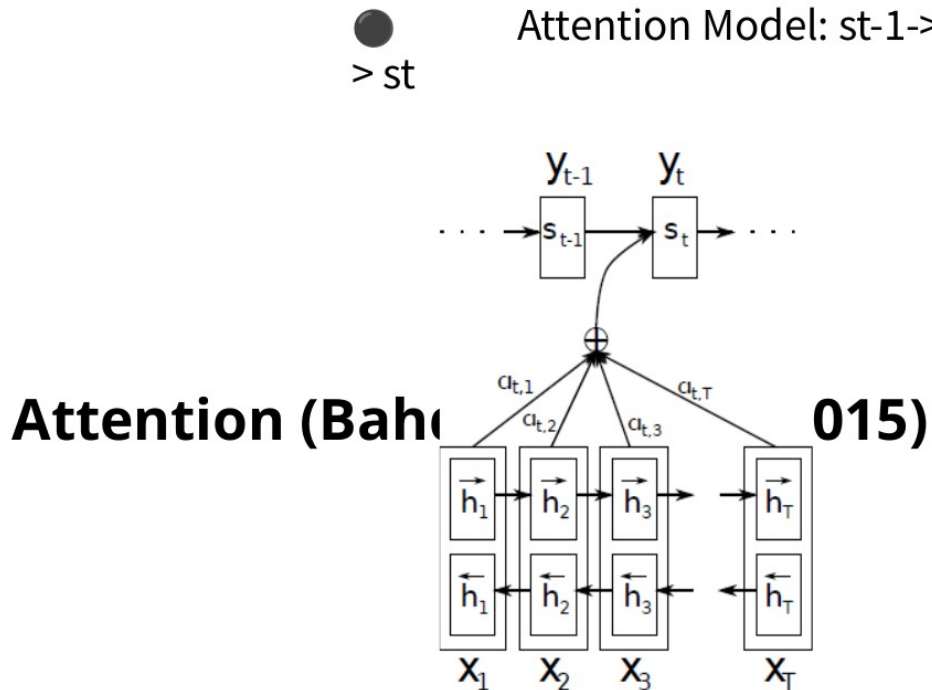
- 소스 시퀀스의 인코딩: 소스 시퀀스의 모든 정보를 포착해야 한다. Bottleneck 이슈

Attention 기초



▶ 디코더가 매 스텝 '원문의 어디를 볼지' 가중(attention) → 고정벡터 병목 해소. Transformer의 직전 단계.

Attention (Bahdanau et al. 2015) — additive



$$p(y_i | y_1, \dots, y_{i-1}, X) = g(y_{i-1}, s_i, c_i), \quad (4)$$

$$s_i = f(s_{i-1}, y_{i-1}, c_i).$$

$$c_i = \sum_{j=1}^{T_x} \alpha_{ij} h_j. \quad (5)$$

$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{k=1}^{T_x} \exp(e_{ik})}, \quad (6)$$

$$e_{ij} = a(s_{i-1}, h_j)$$

▷ 정렬점수 $e = v^T \tanh(W_1 h + W_2 s) \rightarrow \text{softmax} \rightarrow$ 가중합(context). 학습형 정렬.

Attention (Luong et al. 2015) — dot / global · local

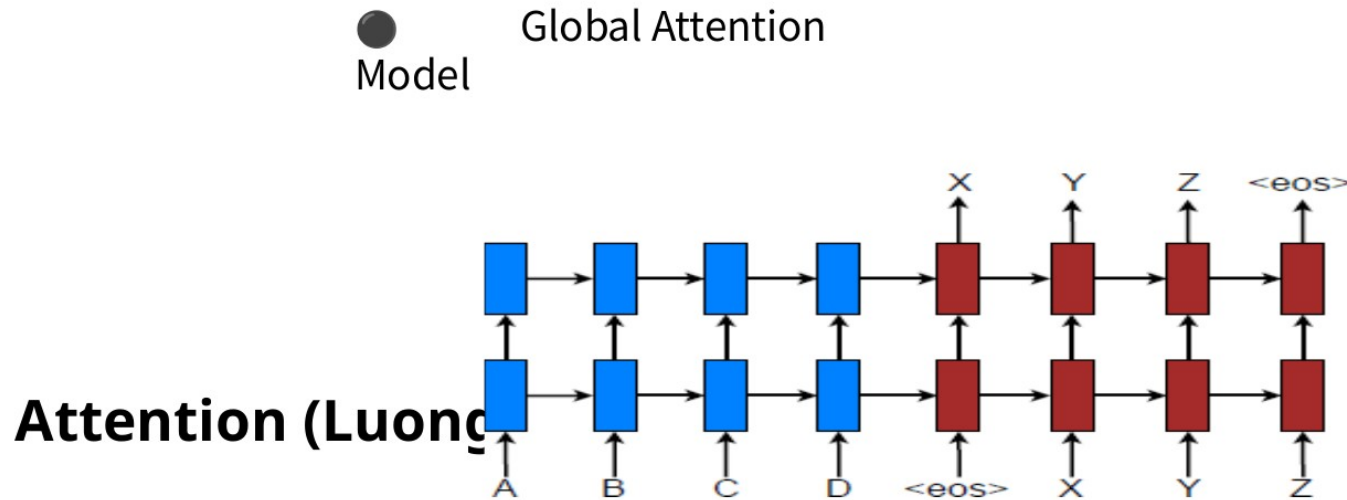
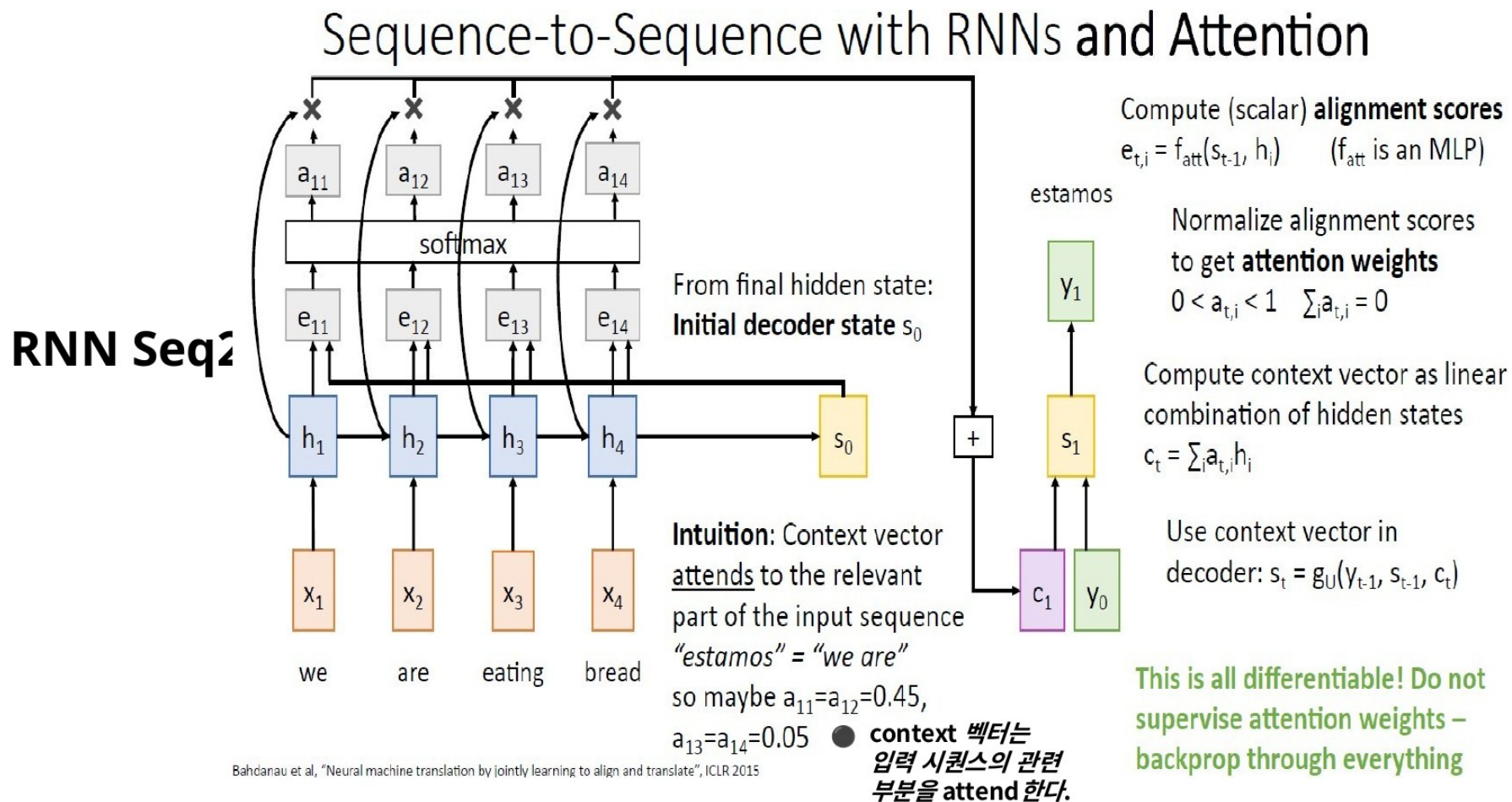


Figure 1: **Neural machine translation** – a stack-
ing recurrent architecture for translating a source
sequence $A \ B \ C \ D$ into a target sequence $X \ Y \ Z$. Here, $\langle \text{eos} \rangle$ marks the end of a sentence.

▷ 내적(dot) 기반 점수로 단순 · 효율화. global vs local attention 구분.

Sequence-to-Sequence with RNNs and Attention



▷ 인코더 은닉상태들 ↔ 디코더가 매 스텝 정렬점수 → softmax → context 생성. 병목 없이 원문 전체를 참조.

attention 가중치 시각화 — 번역 정렬

● 영어를 불어로 번역

Example: English to French translation

RNN Seq2:

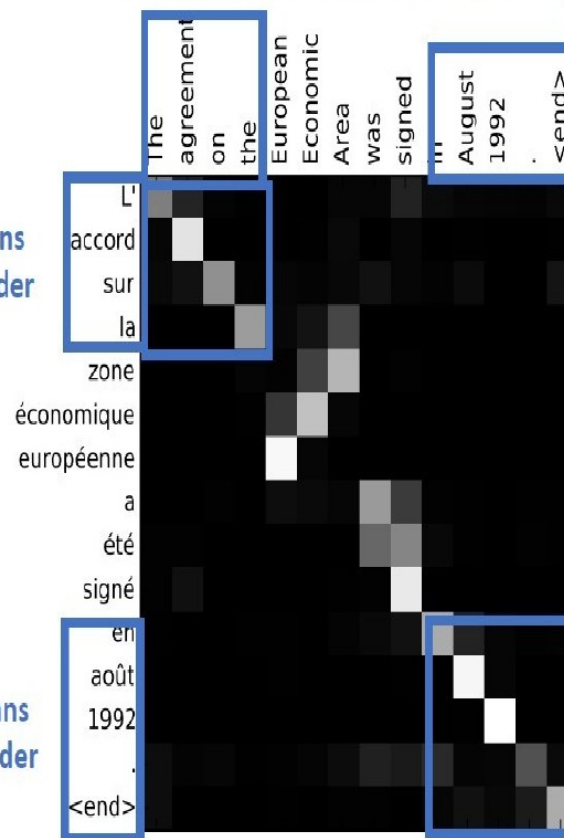
Input: “The agreement on the European Economic Area was signed in August 1992.”

Output: “L'accord sur la zone économique européenne a été signé en août 1992.”

Diagonal attention means words correspond in order

Diagonal attention means words correspond in order

Visualize attention weights $a_{t,i}$



Radanau et al, "Neural machine translation by jointly learning to align and translate", ICLR 2015

▶ 어느 입력 단어에 얼마나 주목했는지 히트맵으로. 대각선=순차 대응, 오프대각=어순 재배열(불어↔영어).

Attention Layer — Query · Key · Value 일반화

Attention Layer

Inputs:

Query vectors: **Q** (Shape: $N_Q \times D_Q$)

Input vectors: **X** (Shape: $N_X \times D_X$)

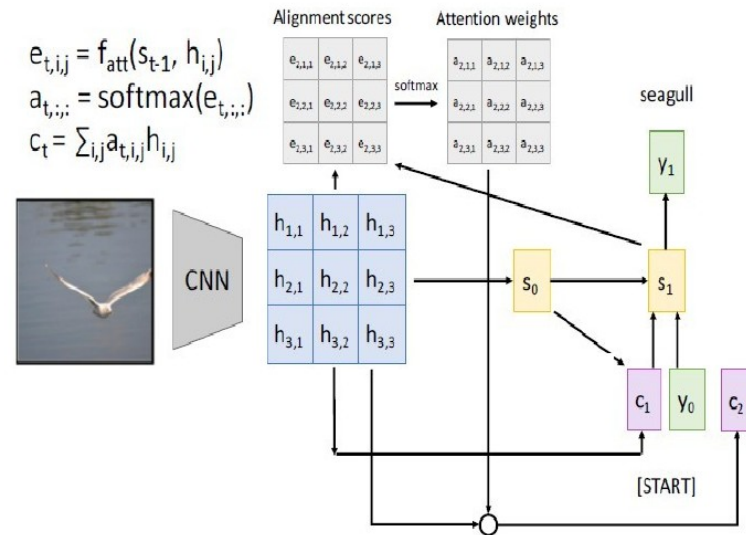
Attention

Computation:

Similarities: $E = QX^T$ (Shape: $N_Q \times N_X$) $E_{i,j} = Q_i \cdot X_j / \sqrt{D_Q}$

Attention weights: $A = \text{softmax}(E, \text{dim}=1)$ (Shape: $N_Q \times N_X$)

Output vectors: $Y = AX$ (Shape: $N_Q \times D_X$) $Y_i = \sum_j A_{i,j} X_j$



Changes:

- Use dot product for similarity
- Multiple **query** vectors

● 여러개의
Query 벡터를 사용한다.

▶ 검색어(Q)로 색인(K)을 훑어 관련 내용(V)을 가중합. 이 일반화가 self-attention으로.

self-attention 핵심 수식 — $Q \cdot K \cdot V$

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k}) V$$

- Query=찾는 것, Key=색인, Value=내용 (도서관 비유)
- $Q \cdot K$ 유사도로 '어느 토큰에 주목할지' → softmax
- $\div \sqrt{d_k}$: 내적이 커져 softmax 포화(기울기 소실)되는 것 방지
- 모든 토큰이 서로를 '동시에' 참조 → 병렬 · 장거리 $O(1)$

쉽게

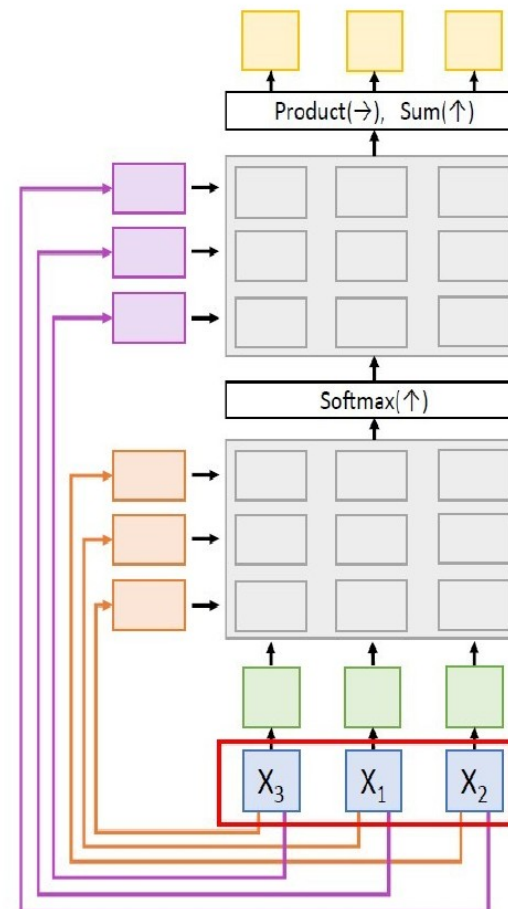
검색어(Q)로 색인(K)을 뒤져 관련 내용(V)을 섞어 요약 — 그게 attention. RNN처럼 순서를 기다리지 않아 빠르고 먼 단어도 직접 참조. 부담은 $O(n^2)$ (2교시 긴 문맥 · RAG로 연결).

Self-Attention Layer

Self-Attention Layer

Inputs:Input vectors: X (Shape: $N_X \times D_X$)Key matrix: W_K (Shape: $D_X \times D_Q$)Value matrix: W_V (Shape: $D_X \times D_V$)Query matrix: W_Q (Shape: $D_X \times D_Q$)

자기 주의 (Self-Attention)

Computation:Query vectors: $Q = XW_Q$ Key vectors: $K = XW_K$ (Shape: $N_X \times D_Q$)Value Vectors: $V = XW_V$ (Shape: $N_X \times D_V$)Similarities: $E = QK^T$ (Shape: $N_X \times N_X$) $E_{i,j} = Q_i \cdot K_j / \text{sqrt}(D_Q)$ Attention weights: $A = \text{softmax}(E, \text{dim}=1)$ (Shape: $N_X \times N_X$)Output vectors: $Y = AV$ (Shape: $N_X \times D_V$) $Y_i = \sum_j A_{i,j} V_j$ Consider **permuting**
the input vectors:입력벡터의
순열을 고려

▶ 입력 벡터마다 자기 자신의 Q/K/V를 만들어 서로 참조. 순서 정보는 positional encoding으로 주입.

Masked Self-Attention — '미래 훑쳐보기' 차단

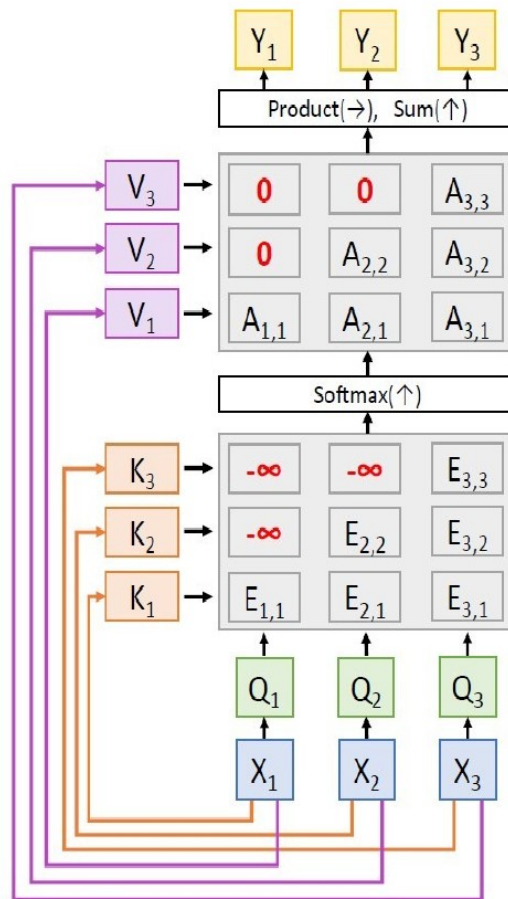
Masked Self-Attention Layer

Don't let vectors "look ahead" in the sequence

● “선견” 편향을 막기 위해 마스크를 사용

Inputs:Input vectors: X (Shape: $N_x \times D_x$)Key matrix: W_K (Shape: $D_x \times D_Q$)Value matrix: W_V (Shape: $D_x \times D_V$)Query matrix: W_Q (Shape: $D_x \times D_Q$)

마스크 자기

Computation:Query vectors: $Q = XW_Q$ Key vectors: $K = XW_K$ (Shape: $N_x \times D_Q$)Value Vectors: $V = XW_V$ (Shape: $N_x \times D_V$)Similarities: $E = QK^T$ (Shape: $N_x \times N_x$) $E_{i,j} = Q_i \cdot K_j / \text{sqrt}(D_Q)$ Attention weights: $A = \text{softmax}(E, \text{dim}=1)$ (Shape: $N_x \times N_x$)Output vectors: $Y = AV$ (Shape: $N_x \times D_V$) $Y_i = \sum_j A_{i,j} V_j$ 

▶ 생성(디코더)에서는 미래 토큰을 못 보게 마스크. GPT류 자기회귀 생성의 핵심.

Multi-head Self-Attention — 여러 관점 병렬

Multihead Self-Attention Layer

Use H independent
“Attention Heads” in parallel

Inputs:

Input vectors: X (Shape: $N_X \times D_X$)

Key matrix: W_K (Shape: $D_X \times D_Q$)

Value matrix: W_V (Shape: $D_X \times D_V$)

Query matrix: W_Q (Shape: $D_X \times D_Q$)

멀티헤드 자

Computation:

Query vectors: $Q = XW_Q$

Key vectors: $K = XW_K$ (Shape: $N_X \times D_Q$)

Value Vectors: $V = XW_V$ (Shape: $N_X \times D_V$)

Similarities: $E = QK^T$ (Shape: $N_X \times N_X$) $E_{i,j} = Q_i \cdot K_j / \text{sqrt}(D_Q)$

Attention weights: $A = \text{softmax}(E, \text{dim}=1)$ (Shape: $N_X \times N_X$)

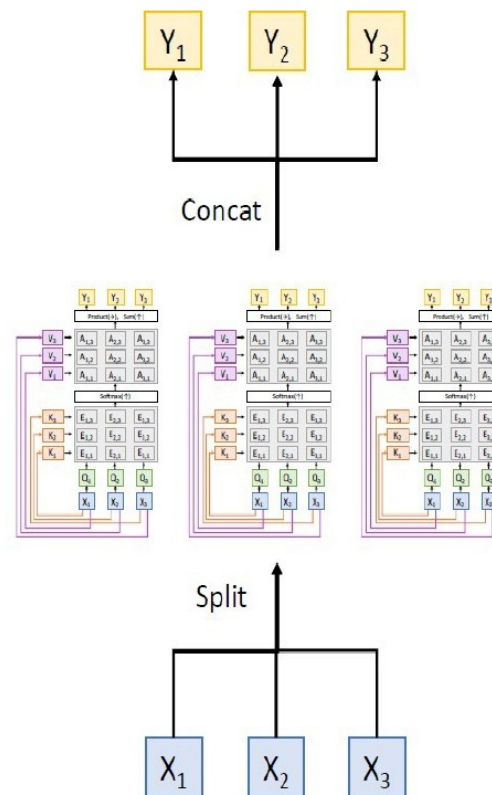
Output vectors: $Y = AV$ (Shape: $N_X \times D_V$) $Y_i = \sum_j A_{i,j} V_j$

H개의 독립
attention head를
병렬로 사용

Hyperparameters:

Query dimension D_Q

Number of heads H



▶ H개 head가 각각 다른 관계(문법·의미·장거리)를 병렬 포착 → concat. 차원을 H로 나눠 총 연산은 single-head와 비슷.

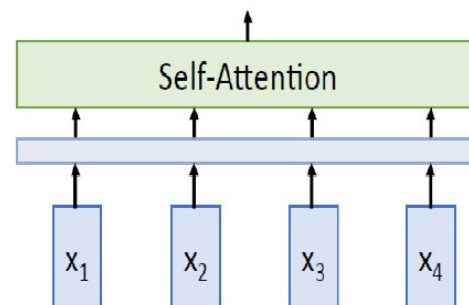
The Transformer — attention으로 병렬·장거리

The Transformer

트랜스포머

- 모든 벡터가 상호 영향

All vectors interact
with each other



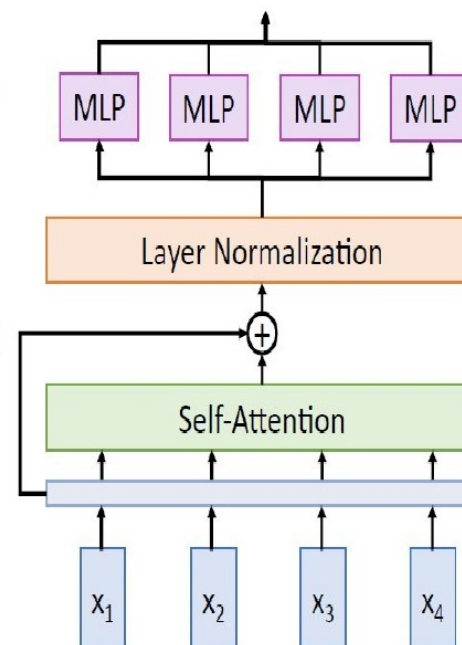
▶ RNN 순차 처리를 self-attention으로 대체 → 병렬 학습 + 장거리 $O(1)$. 이 블록을 깊게 쌓고 인터넷 규모 학습 = GPT/BERT/LLM.

Transformer 블록 — residual · LayerNorm

트랜스포머

Recall **Layer Normalization**:Given $h_1, \dots, h_N$ (Shape: D)scale: γ (Shape: D)shift: β (Shape: D) $\mu_i = (1/D) \sum_j h_{i,j}$ (scalar) $\sigma_i = (\sum_j (h_{i,j} - \mu_i)^2)^{1/2}$ (scalar) $z_i = (h_i - \mu_i) / \sigma_i$ $y_i = \gamma * z_i + \beta$

Ba et al, 2016

● 각 벡터에 대해 MLP
독립적으로 적용MLP independently
on each vectorResidual connection
All vectors interact
with each other

▶ residual(skip)=그래디언트 고속도로, LayerNorm=층 분포 안정화. 두 장치 덕에 수십~수백 층을 안정적으로 쌓는다.

Adding Attention → Self-Attention → Transformer

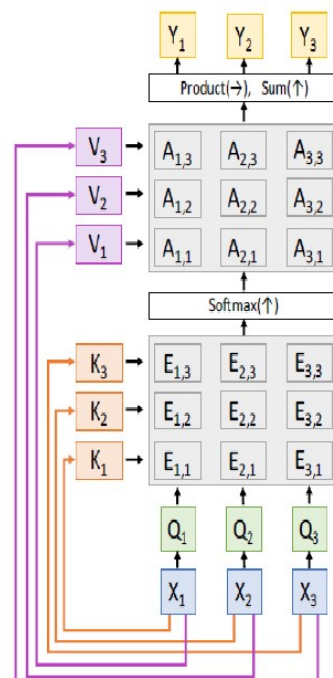
● RNN+attention -> Self-Attention -> Transformer(attention 만 사 용)

Adding **Attention** to RNN models lets them look at different parts of the input at each timestep

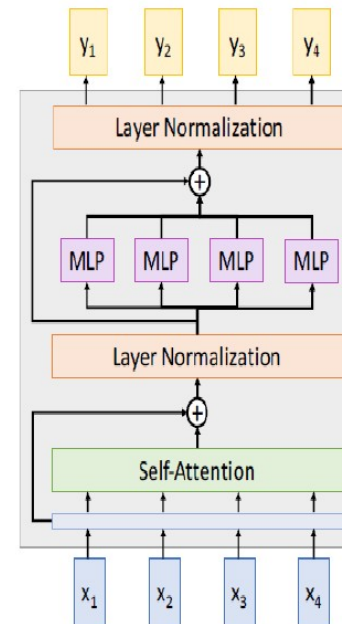


A dog is standing on a hardwood floor.

Generalized **Self-Attention** is new, powerful neural network primitive



Transformers are a new neural network model that only uses attention

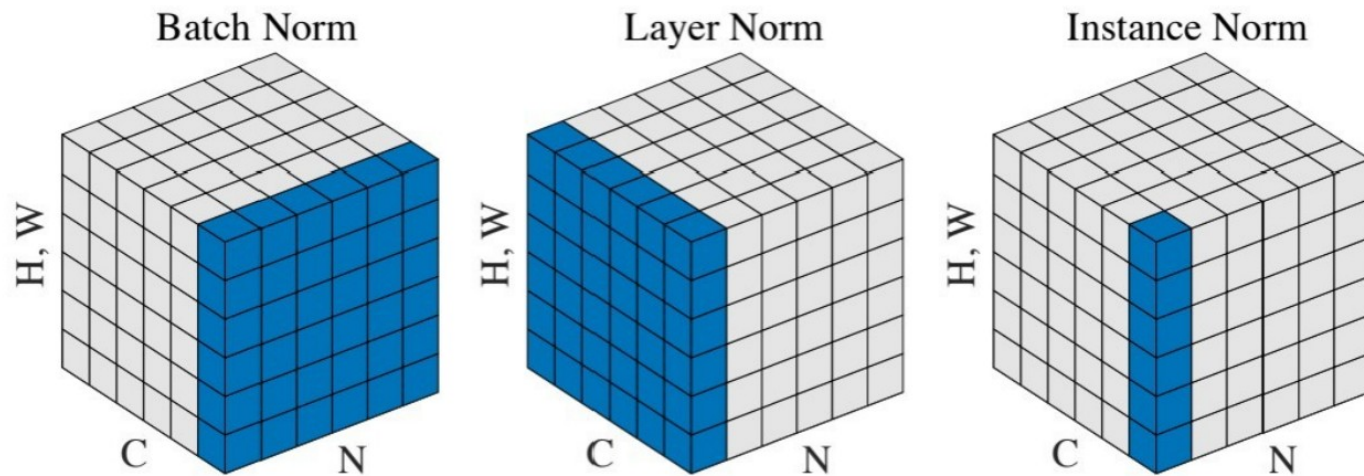


▷ RNN에 attention을 붙이는 단계 → self-attention → 순수 Transformer. 세 단계의 진화를 한 장으로.

정규화 비교 — Batch / Layer / Instance / Group

Comparison of Normalization Layers

Normalizati



Wu and He, "Group Normalization", ECCV 2018

▶ Transformer는 토큰별 LayerNorm 채택(batch 크기·순서에 무관). 최근 LLM은 Pre-LN(서브층 앞 정규화)이 표준.

The Transformer

Transformer Block:

Input: Set of vectors x

Output: Set of vectors y

트랜스포머

Self-attention is the only interaction between vectors!

Layer norm and MLP work independently per vector

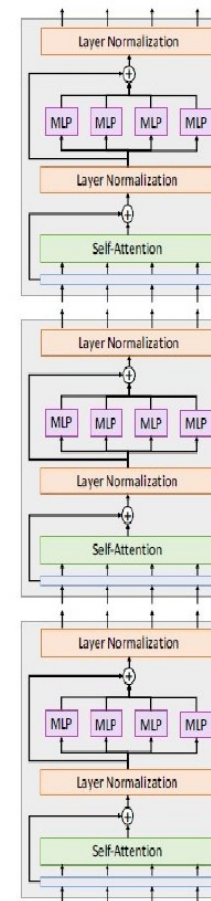
Highly scalable, highly parallelizable

● 트랜스포머는 일련의 트랜스포머 블록이다.

A **Transformer** is a sequence of transformer blocks

Vaswani et al:

12 blocks, $D_Q=512$, 6 heads



▶ 대규모 비지도 사전학습으로 범용 표현 습득 → 소규모 라벨로 과제 적응. 실무는 프롬프팅→PEFT(LoRA)→full 순으로 필요만큼.

Transformers, GPT-2, and BERT

Transformers, GPT-2, and BERT

1. A transformer uses Encoder stack to model input, and uses Decoder stack to model output (using input information from encoder side).
2. But if we do not have input, we just want to model the “next word” , we can get rid of the Encoder side of a transformer and output “next word” one by one. This gives us GPT.
3. If we are only interested in training a language model for the input for some other tasks, then we do not need the Decoder of the transformer, that gives us BERT.

▶ 같은 Transformer, 다른 사용: GPT=decoder(생성), BERT=encoder(이해). 사전학습 목적함수가 용도를 가른다.

GPT 학습 — 비지도(사전학습=미세조정 분리 없음)

GPT Training

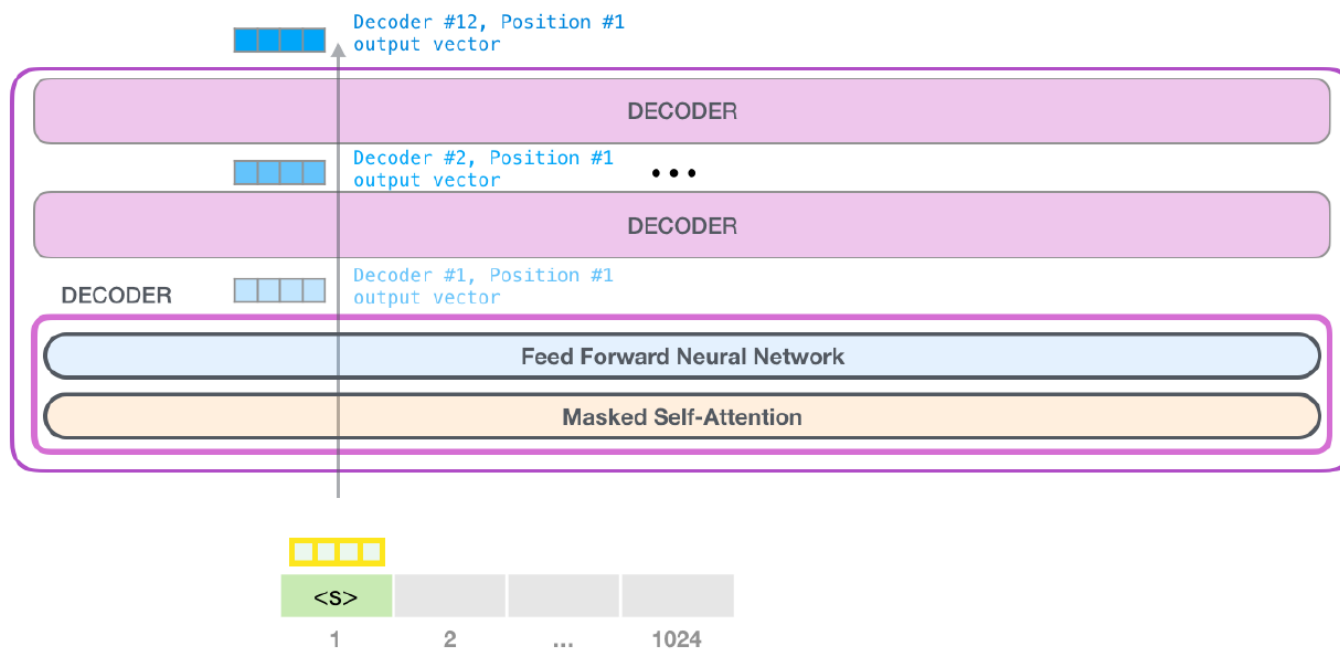
GPT-2 uses **unsupervised** learning approach to training the language model.

There is no custom training for GPT-2, no separation of pre-training and fine-tuning like BERT.

▷ GPT-2는 대규모 텍스트로 '다음 토큰 예측'만 학습. 별도 태스크 학습 없이 프롬프트로 다양한 작업 수행.

GPT-2 생성 — top-k 샘플링

GPT-2 has a parameter top-k, so that we sample words from top k (highest probability from softmax) words for each output

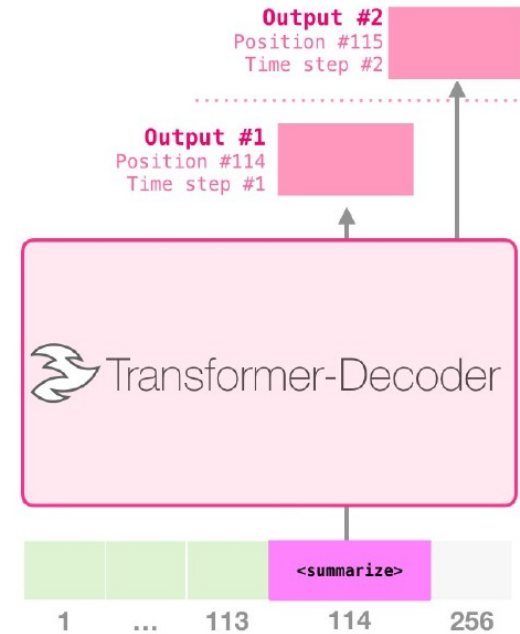


▶ 매 스텝 상위 k개 후보에서 샘플링(다양성 · 안정성 균형). temperature · top-p와 함께 생성 제어.

GPT-2 Application: Summarization

Training Dataset

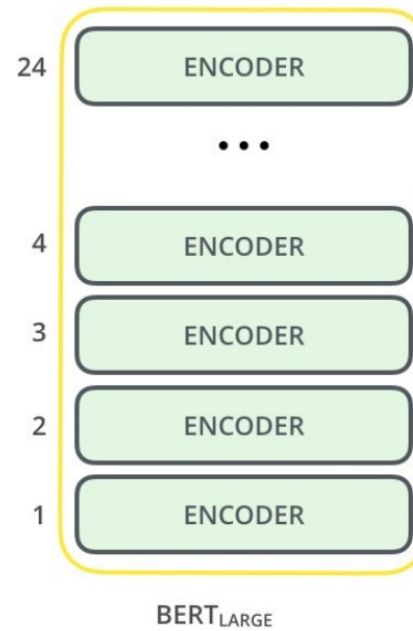
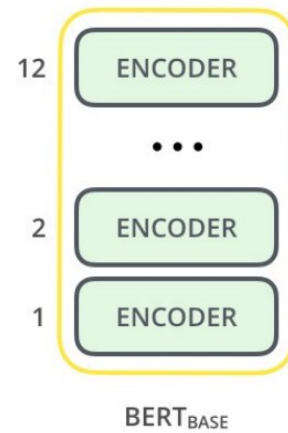
Article #1 tokens	<summarize>	Article #1 Summary
Article #2 tokens	<summarize>	Article #2 Summary
Article #3 tokens	<summarize>	Article #3 Summary



▶ 별도 학습 없이 프롬프트만으로 번역·요약 수행(in-context). 대형화가 만든 사용법.

BERT — 양방향 인코더 표현

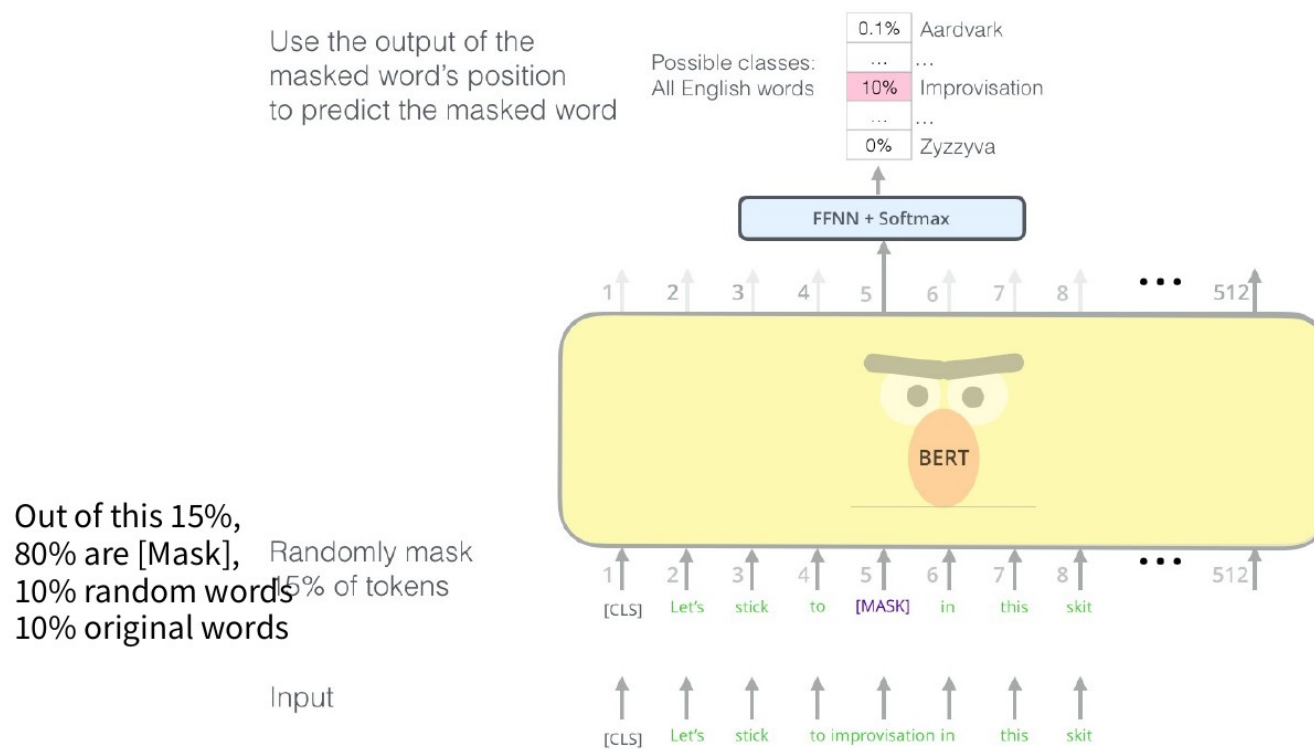
BERT (Bidirectional Encoder Representation from Transformers)



▶ 양쪽 문맥을 동시에 보는 인코더(이해·분류에 강함). 입력 차원 512, [CLS]/[SEP] 토큰 구조.

BERT 사전학습 ① Masked LM

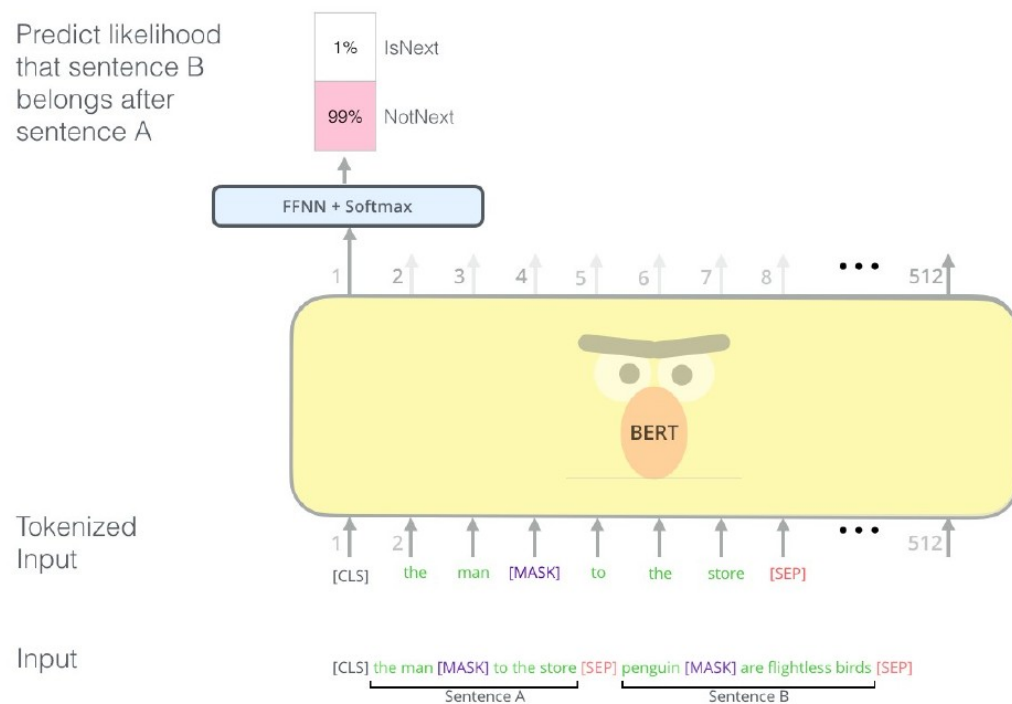
BERT Pretraining Task 1: masked words



▶ 입력의 15%를 가리고 복원(양방향 문맥 활용). GPT의 '다음 토큰'과 대비되는 '가린 토큰' 예측.

BERT 사전학습 ② Next Sentence Prediction

BERT Pretraining Task 2: two sentences

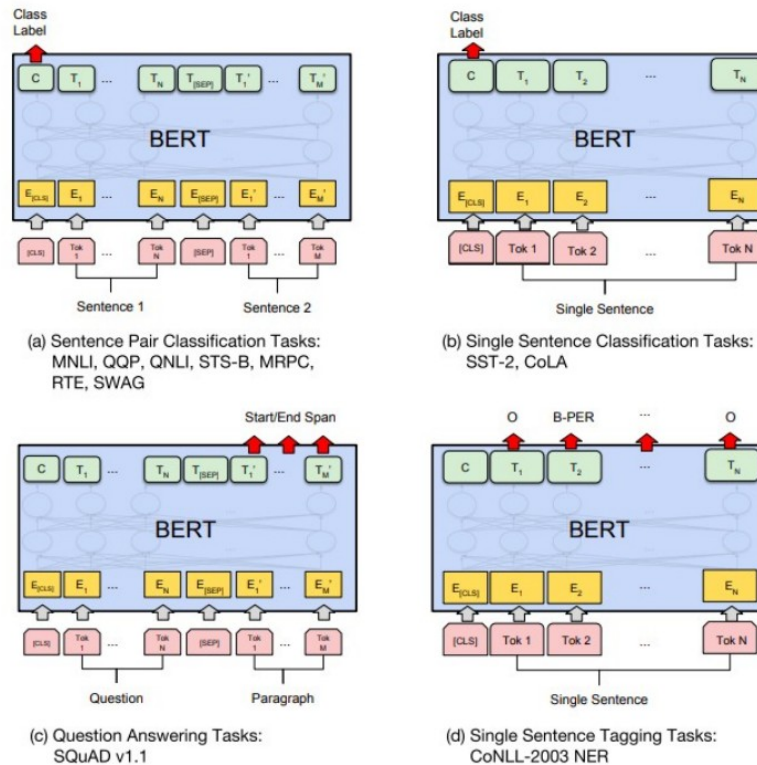


▶ 두 문장이 이어지는지 판별(문장 관계 학습). 50% 진짜 후속 / 50% 무작위.

BERT 미세조정 — 다양한 다운스트림

Fine-tuning BERT for other specific tasks

MNLI
 QQP (Quora Question Pairs)
 Semantic equivalence)
 QNLI (NL inference dataset)
 STS-B (texture similarity)
 MRPC (paraphrase, Microsoft)
 RTE (textual entailment)
 SWAG (commonsense inference)
 SST-2 (sentiment)
 CoLA (linguistic acceptability)
 SQuAD (question and answer)

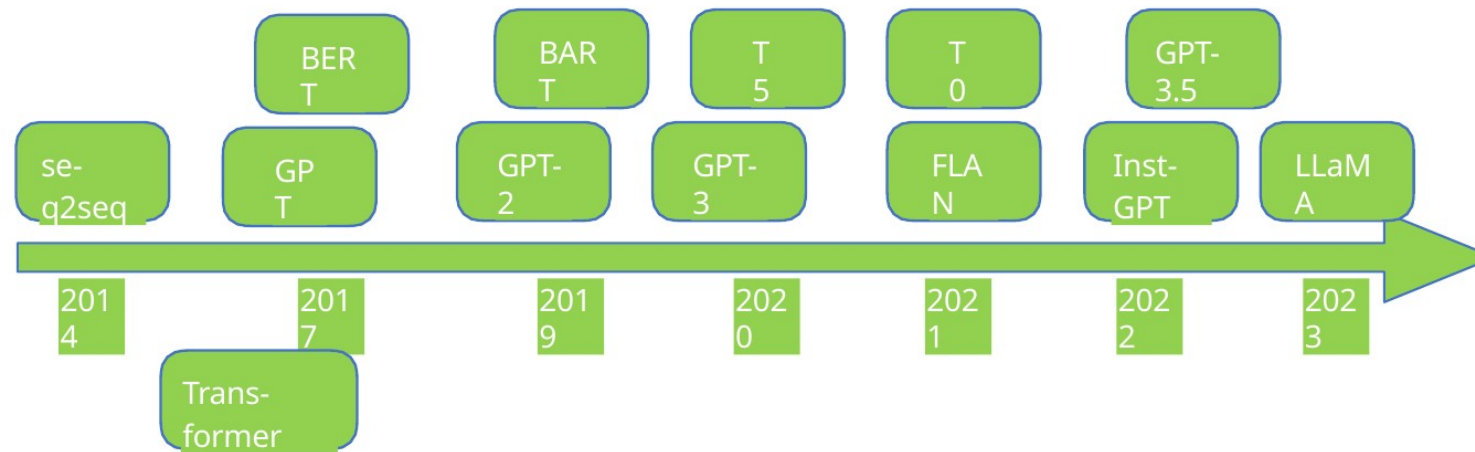


SST (Stanford
 sentiment tree-
 bank): 215k
 phrases with fine-
 grained senti-
 ment labels in the
 parse trees of 11k
 sentences.

▷ [CLS] 표현 위에 작은 head를 얹어 분류 · NER · QA 등으로 적용. 사전학습 → 미세조정 패러다임의 대표.

모델 계보 타임라인 — BERT·GPT·T5·FLAN

models in this class: Timeline



▶ 2017 Transformer 이후 encoder(BERT) · decoder(GPT) · enc-dec(T5/BART) 세 갈래. 최근 대형 LLM은 decoder-only 우세.

T5 — 모든 과제를 text-to-text로

T5: outline

- Text to Text model



Raffel et al., Exploring the Limits of Transfer Learning with a Unified Text-to-text Transformer, 2019

▶ 번역·요약·분류를 전부 '텍스트→텍스트'로 통일. span corruption으로 사전학습.

정렬(alignment) — InstructGPT / RLHF

Reinforcement Learning

Reinforcement Learning: 경험을 통해서 더 나은 결정을 내리도록 학습하는 방식

- Agent: 학습하고자 하는 모델 State: Agent 의 현재 상태

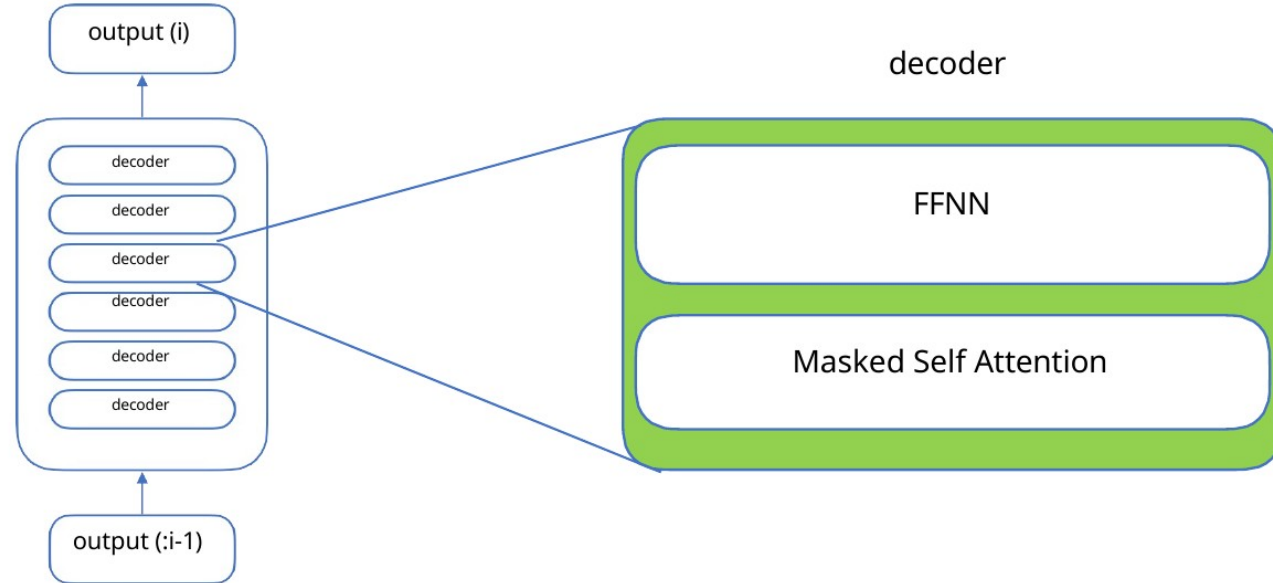
Environment: Agent와 상호작용하는 환경 Reward: 환경이 Agent에게 주는 피드백

▶ ① SFT(사람 시연) → ② 보상모델(선호 순위) → ③ RL(PPO, 보상 ↑ + KL 제약). 목표: 유용 · 정직 · 무해(3H). 최근엔 DPO로 간소화.

LLaMA / ChatGPT — decoder-only Transformer

Touvron et al., LLaMA: Open and Efficient Foundation Language Models, 2023

Architecture: Decoder only Transformer



▶ 현대 대형 LLM의 주류 = decoder-only. 이 구조 위에 RoPE 위치인코딩 · GQA · 긴 문맥 등이 얹힌 것이 GPT-5 · Claude · Gemini · Llama.

LLaMA vs GPT — 데이터 & 학습 단계

Touvron et al., LLaMA: Open and Efficient Foundation Language Models, 2023

Data & Training

LLaMA

Dataset	Sampling prop.	Epochs	Disk size
CommonCrawl	67.0%	1.10	3.3 TB
C4	15.0%	1.06	783 GB
Github	4.5%	0.64	328 GB
Wikipedia	4.5%	2.45	83 GB
Books	4.5%	2.23	85 GB
ArXiv	2.5%	1.06	92 GB
StackExchange	2.0%	1.03	78 GB

Unsupervised pre-training

GPT

Dataset	Quantity (tokens)
Common Crawl (filtered)	410 billion
WebText2	19 billion
Books1	12 billion
Books2	55 billion
Wikipedia	3 billion

Unsupervised pretraining
+ supervised finetuning
+ reinforcement learning w. human feedback

▶ 비지도 사전학습 → 지도 미세조정(SFT) → 인간 피드백 강화학습(RLHF). 오픈(Llama)·폐쇄(GPT) 모두 이 3단계 골격.

스케일링 법칙 — 왜 '더 크게'가 통했나

$$L(N,D,C) \propto \text{먹법칙 감소} \cdot C \approx 6 \cdot N \cdot D$$

Chinchilla 최적: 토큰 $D \approx$ 파라미터 $N \times 20$ (크기:데이터 균형)

- Kaplan 2020: 손실이 모델·데이터·연산의 먹법칙으로 예측 가능하게 감소
- Chinchilla 2022: 같은 연산이면 '작은 모델+많은 데이터'가 유리 — '데이터가 왕'
- 이 예측 가능성이 GPT-3→현재의 대형화 경쟁을 촉발

쉽게

'모델·데이터·연산을 키우면 손실이 예측 가능하게 준다'가 수식으로 밝혀져, 대형화에 투자할 근거가 생겼다. 단 크기만이 아니라 데이터 균형이 핵심(Chinchilla).

범용(프론티어) LLM 지형 — 2026 현행

모델군	2026 현행 · 특징	접근
OpenAI GPT-5 계열	GPT-5.x · 코딩특화 GPT-5.3-Codex · 추론형	API · 폐쇄
Anthropic Claude 5	Opus 5 / Sonnet 5 · Claude for Life Sciences	API · 폐쇄
Google Gemini 3	Gemini 3 Pro · Gemini for Science	API · 폐쇄
오픈 가중치	Llama 4 · DeepSeek(R1) · Qwen3 · Mistral	오픈 · 온프레미스

+심화

2026 흐름: (1) 추론형(reasoning) 기본 탑재, (2) 'agentic' — 모델이 챗봇에서 '도구를 실행하는 워커'로(→3교시). 세부 버전은 빠르게 갱신되므로 2026.9 시점 확인 권장.

PART B

과학 언어모델 (2026)

같은 Transformer, 다른 어휘 — 단백질 서열·분자 SMILES·논문을 '언어'로

범용 LLM vs 과학 언어모델 (2026)

생물학 언어 (B-1)

- 단백질: ESM3(생성) · ESMC(표현) · ESMFold2
- 생의학 텍스트: SciBERT · PubMedBERT · BioGPT
- 치료제/오믹스: TxGemma · scGPT · Geneformer
- 아미노산 20자 '알파벳'
서열 → 구조 · 기능

화학 언어 (B-2)

- 분자 문자열: SMILES · SELFIES
- 표현: ChemBERTa-2 · MolT5 · SMI-TED
- 파운데이션: ChemDFM · ChemLLM · 멀티모달 ChemVLM
- 원자 · 결합을 토큰으로
분자 ↔ 자연어 · 물성 · 생성

왜 과학 특화 LM인가 — Galactica 교훈 (여전히 유효)

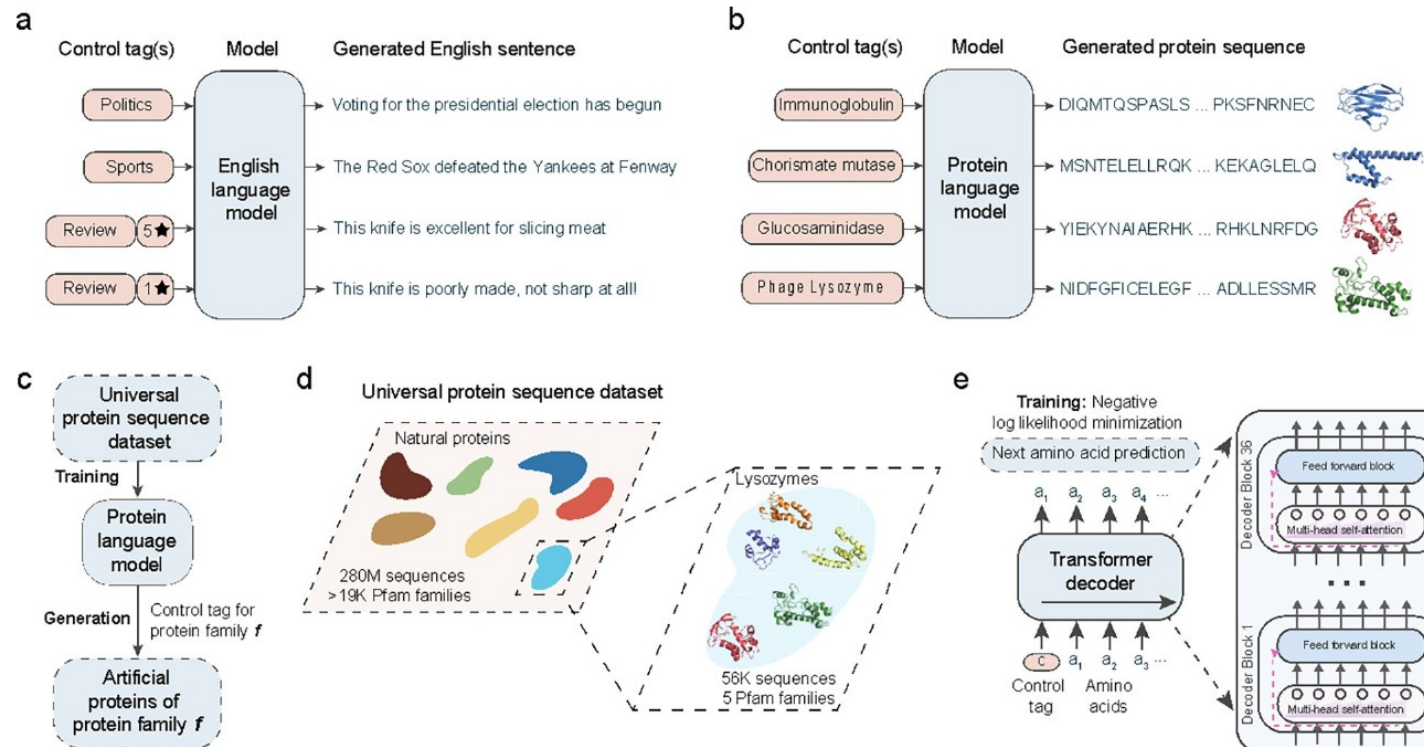
- 일반 LLM은 과학 전문용어·수식·구조식·최신 문헌에 약함
- Galactica(Meta 2022): 과학 전용 LLM 데모 → 공개 3일 만에 철회
- 이유: 그럴듯하나 틀린 출력(환각)·가짜 논문·서지 생성
- 교훈(2026에도): '유창함 ≠ 정확성' → 특화·RAG·검증·에이전트 필요



합정

Galactica 사건은 이 과정의 출발점 — 과학 LLM은 말이 유창해도 틀릴 수 있다. 도메인 특화·근거 제시(RAG)·도구·인간 검증이 필수(2·3교시).

단백질 언어 — Learning the protein language (ESM)



▶ 단백질=아미노산 '알파벳'으로 쓴 문장. 대규모 MLM 학습 → 구조·기능이 임베딩에 창발.

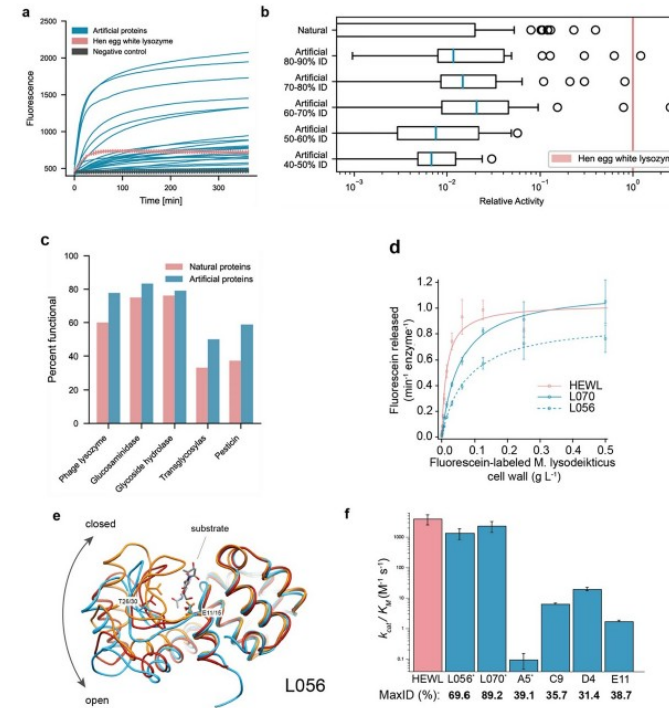
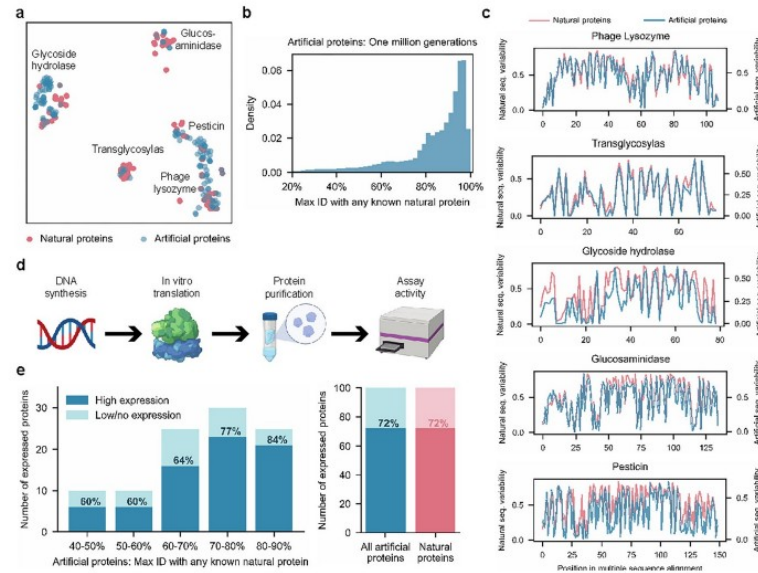
단백질 언어모델 / 구조 — 2026 현행

모델	유형	핵심 (2026)
ESM3	멀티모달 생성	서열·구조·기능 통합 track, open 1.4B, 신규 esmGFP
ESM Cambrian(ESMC)	표현(임베딩)	~2.8B 서열·최대 6B 파라미터, 대규모 표현
ESMFold2	구조 예측	FoldBench서 AF3 대등/상회 주장(확인 권장)
AlphaFold3 / Boltz-2	복합체 구조	AF3(2024)·Boltz-2(2025, 3일차 실습)·Chai-1

+심화

단백질 LM = '표현(ESMC 임베딩)' + '생성(ESM3)'. MSA-free·초고속이 대규모 스캔의 열쇠. 구조 예측은 AF3·Boltz-2·ESMFold2가 경쟁 — 3일차(구조기반)와 직접 연결.

ProGen — 조건부로 기능성 단백질 서열 생성

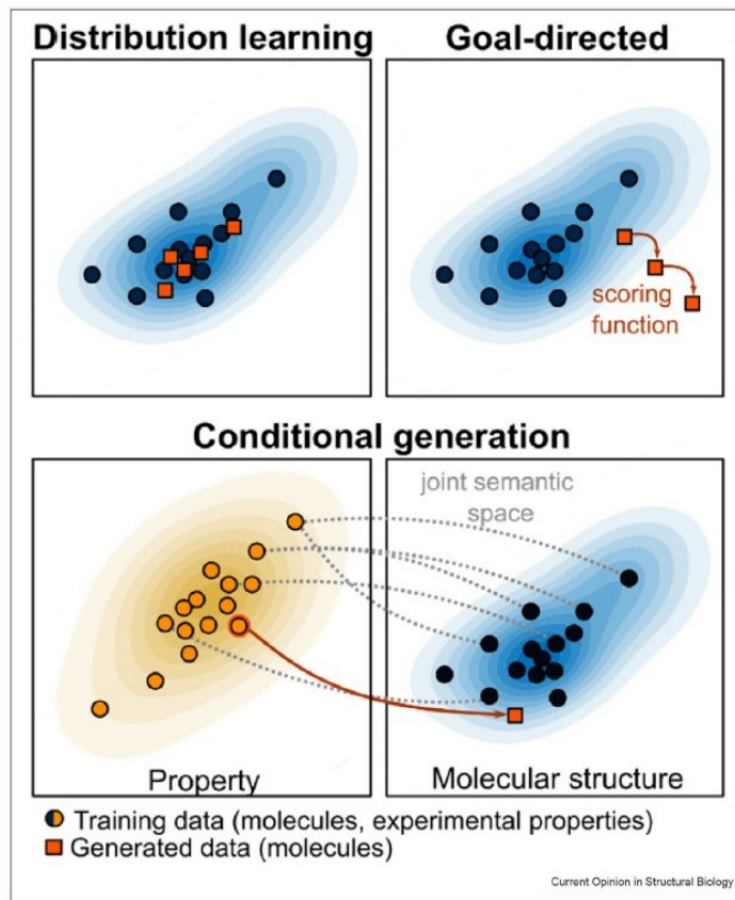


▶ 기능 tag를 조건으로 신규 서열 생성 → 라이소자임이 wet-lab에서 실제 효소로 작동 입증. '이해'를 넘어 '생성'으로.

생의학 텍스트 · 치료제 · 단일세포 (2026)

영역	모델 (2026 현행)	핵심 · 주의
생의학 텍스트	SciBERT · PubMedBERT(이해) · BioGPT(생성)	도메인 from-scratch 우위
치료제 범용	TxGemma(2B/9B/27B, Gemma-2)	Tx-LLM 대체 · TDC 66태스크
치료 에이전트	TxAgent / Agentic-Tx (Gemini 2.0)	도구 · 추론 자율(→3교시)
단일세포	scGPT · Geneformer · scFoundation · CellFM	⚠ 섭동예측: 단순 PCA/baseline 우세 다수 (5교시 재론)

단일세포 LM — scBERT / scGPT (전사체를 '언어'로)



▶ 유전자 발현을 토큰으로 → 세포 유형 주석·섭동 예측. 5교시 멀티오믹스에서 강점·한계를 함께 다룬다.

PART B-2

화학 언어모델 (2026)

분자를 '언어'로 — SMILES · SELFIES · 표현→생성→멀티모달

화학 LM 계열 — ChemBERTa · MolBERT · ChemGPT

1. ChemBERTa:

•논문: "ChemBERTa: Large-Scale Self-Supervised Pretraining of Transformer Models for Molecular Property Prediction"

•특징: RoBERTa 기반의 모델로, SMILES 코드를 학습해 분자 특성 예측에 활용됩니다.

•응용 분야: 약물 특성 예측, 화합물 설계

2. ChemGPT:

•논문: "Generative Pre-trained Transformer (GPT) Models for Molecular Design"

•특징: GPT 모델의 구조를 활용해 화학 데이터에서 새로운 분자를 생성하고 최적화합니다.

•응용 분야: 새로운 분자 생성, 약물 설계

3. MolBART:

•논문: "MolBART: Molecular Representation Learning with Bidirectional AutoRegressive Transformers"

•특징: BART 기반의 모델로, 화합물 변형 및 생성에 탁월한 성능을 보입니다.

•응용 분야: 약물-타겟 상호작용 예측, 화합물 최적화

4. SMILES Transformer:

•논문: "Attention-Based Transformers for Predicting Molecular Properties"

•특징: SMILES 표현을 통해 분자 특성을 예측하고 생성하는 모델로, 다양한 화학적 특성 예측에 활용됩니다.

•응용 분야: 분자 구조 생성, 약물 설계

5. MolFormer:

•논문: "MolFormer: Self-Supervised Transformer-Based Representation Learning for Molecules"

•특징: Microsoft에서 개발한 모델로, 약물-타겟 상호작용 예측과 신약 후보 발굴에 효과적입니다.

•응용 분야: 신약 후보 발굴, 화합물의 생물학적 특성 예측

6. MolT5:

•논문: "MolT5: A Molecular Language Model for Translating Chemical Texts"

•특징: T5 모델을 기반으로 분자 구조 변환과 신약 후보 생성에 활용됩니다.

•응용 분야: 분자 구조 변환, 화합물 라이브러리 확장

▶ SMILES를 토큰으로 자가지도 사전학습 → 분자 임베딩(물성 예측). BERT(이해)/GPT(생성) 구조의 화학판.

화학 언어모델 3세대 — 전용 번역기 → 지시학습 LLM → 멀티모달·프론티어

세대	모델 (실제 HF ID / 계열)	유형	핵심 (2026)
① 2020-22	ChemBERTa-2 · SMI-TED	표현(임베딩)	SMILES 자기지도 → 물성 예측
① 2022	laituan245/molT5-large · BioT5	전용 번역기(T5)	설명↔SMILES 고정 태스크 · 대화 불가 (6일차 실습)
② 2024	OpenDFM/ChemDFM-v1.5-8B	화학 지시학습 LLM	LLaMA-3-8B 기반 · 대화·다중태스크 · AGPL-3.0
② 2024	AI4Chem/ChemLLM-7B-Chat	화학 대화 LLM	InternLM2-7B 기반 · Apache-2.0 · 단계적 추론
② 2023-24	DrugAssist (Llama-2-7B)	대화형 최적화	human-in-loop 물성 조정
③ 2025-26	AI4Chem/ChemVLM-8B	멀티모달(이미지)	구조 이미지 직접 입력 image-text-to-text · Apache-2.0
③ 2025-26	Gemini · GPT-5 (상용 API)	범용 프론티어	화학 전용학습 없이 추론 — 유명분자 강함/신규골격 취약

+심화

세대가 바뀌면 '던질 수 있는 질문'이 바뀐다 — ① 고정 태스크 번역 → ② 대화·다중태스크 → ③ 이미지 입력·범용 추론. 6일차 t042 실습에서 같은 실데나필 문제를 세 세대에 그대로 던져 직접 확인(1-샘플 시연, 벤치마크 아님).

MolGPT — Transformer 디코더로 분자 생성

Model	Sequence-to-sequence (%)			Discriminative (Mean R^2)	
	Direct	Retro	Mol opt	Mol prop	Bioactivity
Random	91.1	50.8	73.1	0.680	0.480
Mask	91.2	52.1	75.0	0.843	0.603
Augment	91.1	51.8	74.3	0.848	0.606
Combined	91.8	53.6	72.2	0.857	0.631

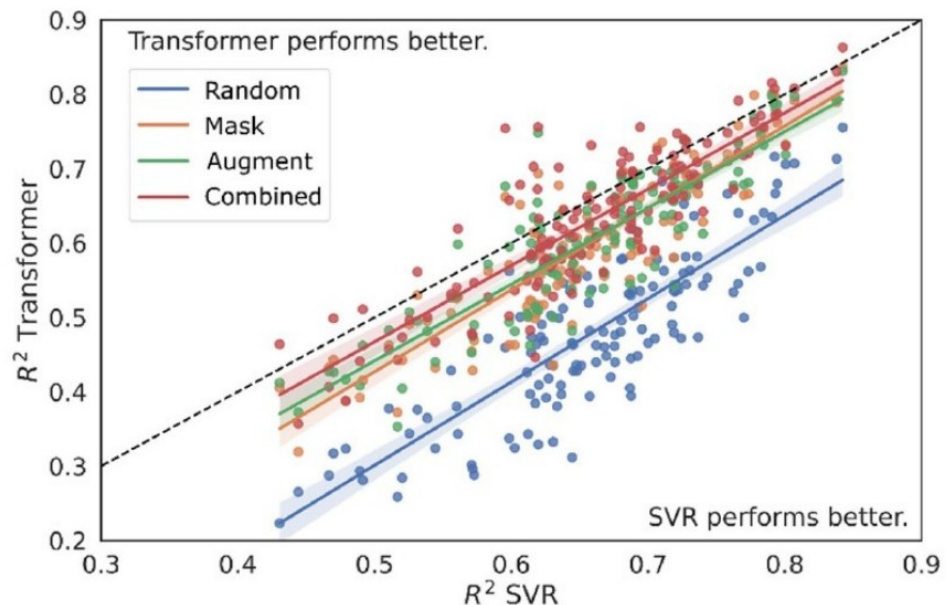
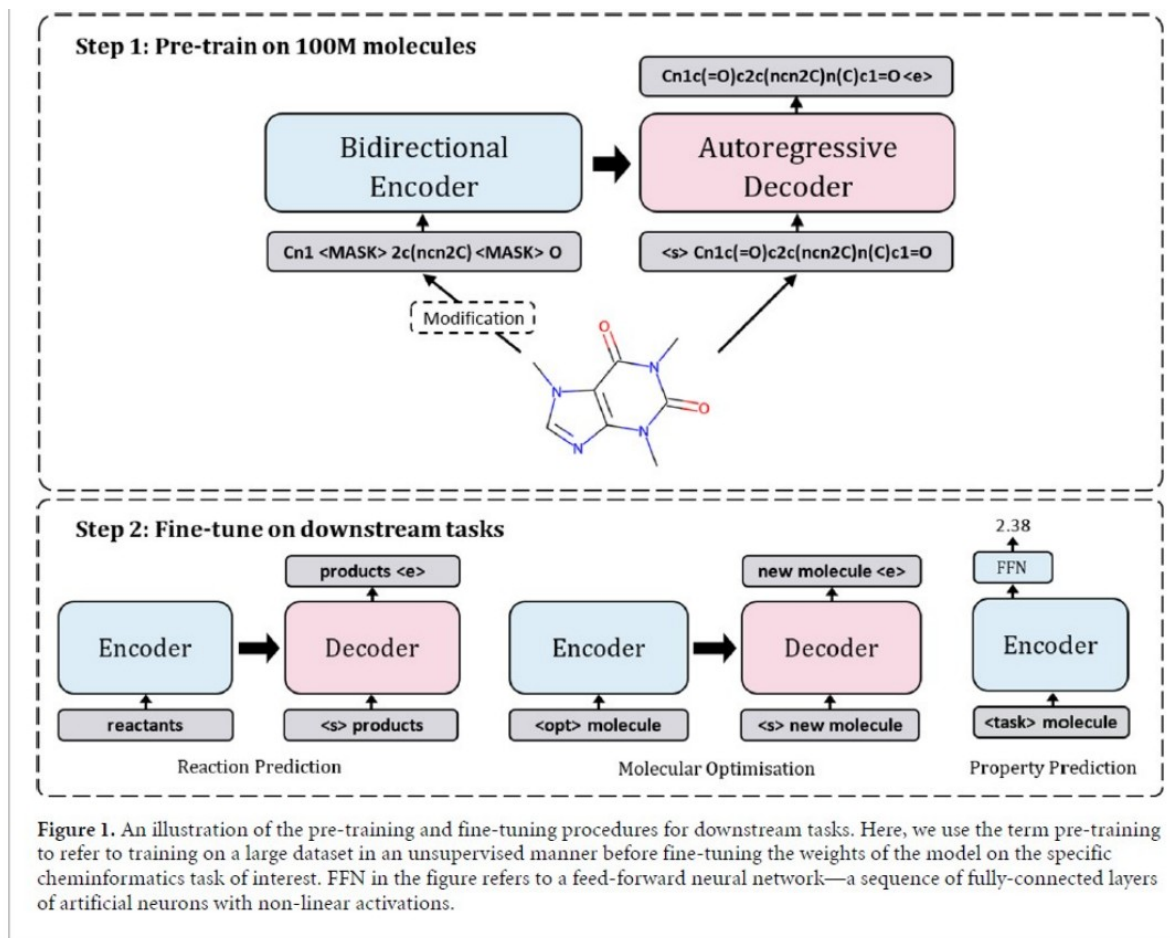


Figure 3. Comparison of the performance of Chemformer models with that of an SVR baseline across 133 bioactivity prediction tasks. Each dot corresponds to the bioactivity prediction result for a single gene. If the dot is above the dashed line the Chemformer model is a better predictor for that gene.

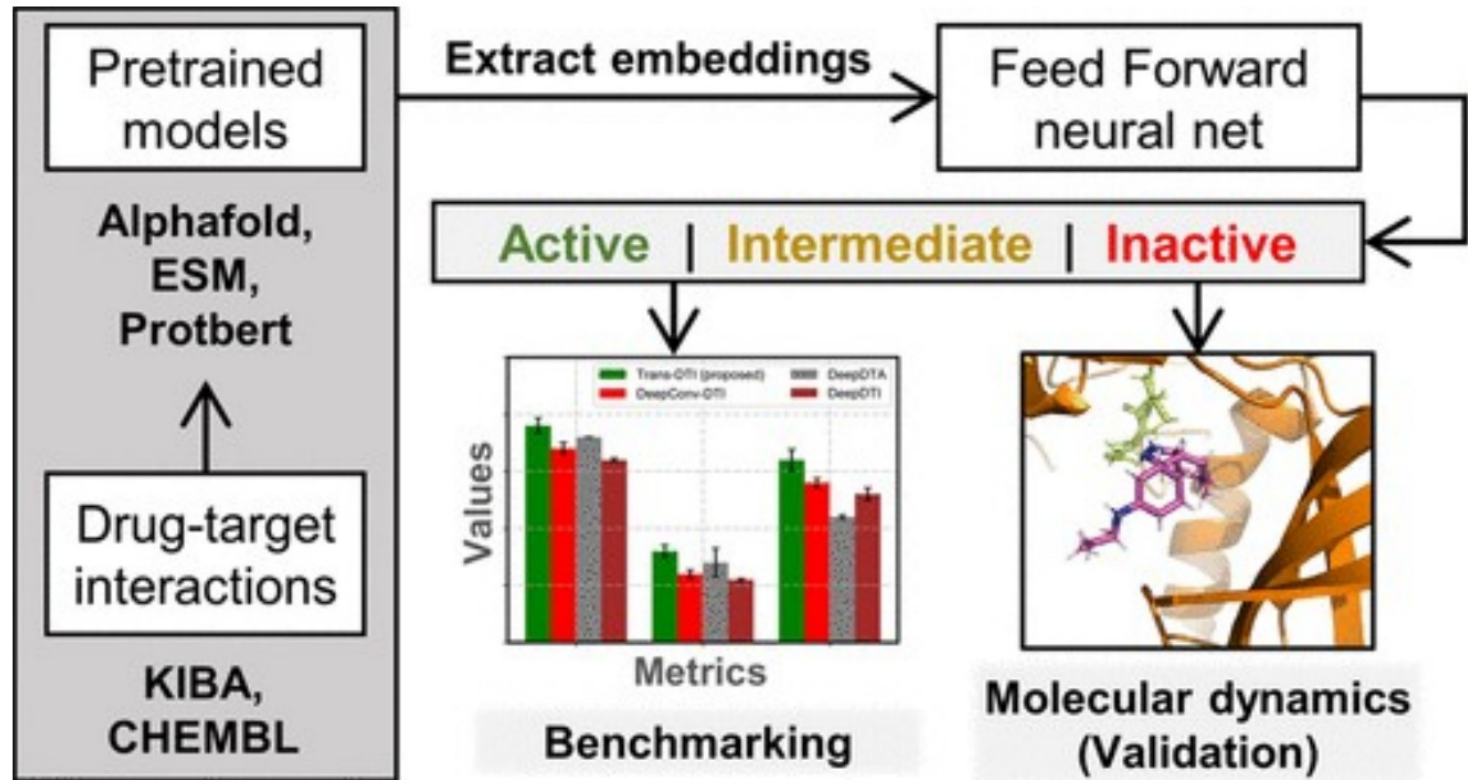
▶ GPT식 자기회귀로 SMILES를 한 토큰씩 생성. 스캐폴드·물성 조건부 생성 가능(4일차 생성모델과 연결).

ChemFormer — 사전학습 Transformer(합성 · 물성)



▶ 반응 예측 · 역합성 · 물성으로 미세조정되는 화학 특화 seq2seq Transformer.

사전학습 표현 전이 — DTI/DTA 예측



▶ AlphaFold · ESM · ProtBert 임베딩을 특징으로 추출 → 약물-표적 상호작용 예측. 사전학습 표현의 '만능키' 활용 예.

LLM 파인튜닝 — 화학 문헌 텍스트 마이닝

Chemical
Science



EDGE ARTICLE



Cite this: *Chem. Sci.*, 2024, 15, 10600

All publication charges for this article have been paid for by the Royal Society of Chemistry

Fine-tuning large language models for chemical text mining†

Wei Zhang,^{‡ab} Qinggong Wang,^{‡c} Xiangtai Kong,^{ab} Jiacheng Xiong,^{ab} Shengkun Ni,^{ab} Duanhua Cao,^{ad} Buyang Niu,^{ab} Mingan Chen,^{aef} Yameng Li,^g Runze Zhang,^{ab} Yitian Wang,^{ab} Lehan Zhang,^{ab} Xutong Li,^{ab} Zhaoping Xiong,^g Qian Shi,^f Ziming Huang,^h Zunyun Fu^{*a} and Mingyue Zheng^{id*abc}

▶ 논문에서 반응물·촉매·용매·조건·NMR을 자동 추출 → 실험노트·DB 구축 자동화(6일차 실습 에이전트와 연결).

과학 특화 상용 — Claude for Life Sciences · Gemini for Science (2026)

Claude for Life Sciences

- Anthropic 생명과학 특화(Claude Sonnet/Opus 계열)
- 문헌리뷰→가설→데이터분석→규제문서 초안
- MCP 커넥터로 랩 도구·DB 연결
Benchling·PubMed 등

Gemini for Science · TxGemma

- Google 과학 특화 + TxGemma(치료제)
- AI co-scientist(가설)·TxAgent(치료 태스크)
- agentic — 도구 실행형
3교시에서 에이전트로 상세

+심화

범용 LLM이 '과학 전용 제품'으로 진화 — 연구 전 과정 지원 + MCP로 실험실 도구 연결. 6일차 실습(Claude Code 과학 에이전트)의 배경이자 3교시 심화 주제.

PART C

정리

일반 LLM ↔ 과학 LM · 2026 지형 · 2교시로의 연결

일반 LLM vs 과학 LM — 핵심 대비

일반(범용) LLM (A)

- 어휘: 자연어 토큰(BPE)
- 데이터: 웹·책·코드(조 단위)
- 목표: 다음 토큰 예측·정렬(RLHF)
- 강점: 범용 언어·추론·대화
약점: 도메인 정밀·최신성

과학 LM (B)

- 어휘: 서열/SMILES/전문용어
- 데이터: PubMed·UniProt·분자 DB
- 목표: 도메인 MLM/AR + 임베딩 전이
- 강점: 도메인 정밀·표현
약점: 최신성·환각·검증

+심화

같은 Transformer 원리(A) 위에서 '어휘·데이터·목표'만 도메인에 맞춘 것이 과학 LM(B). 어느 쪽도 사실성은 보장 못 함 → RAG·도구·검증·에이전트가 공통 해법(2·3교시).

1교시 정리

1

일반 LLM = 자기회귀 밀도모델 — RNN→Transformer→GPT/BERT (박혜진 교안 도식)

2

self-attention($QK^T/\sqrt{d_k}$) · multi-head · residual/LayerNorm이 핵심 장치

3

생물학 LM 2026: ESM3(생성) · ESMC(표현) · ESMFold2 · TxGemma · scGPT

4

화학 LM 3세대: MolT5(전용) · ChemDFM/ChemLLM(지시학습) · ChemVLM(멀티모달)

5

Galactica 교훈(유창 ≠ 정확) → RAG · 검증 · 에이전트

다음(2교시): 일반 LLM 활용 + 환각 통제 — 프롬프트 엔지니어링 · RAG · 출처 인용 표준 · 사실성 검증 체크리스트

주요 출처 (1교시)

- 원리 도식: 박혜진 교안(202507_바이오강의교안_full3_part2) 슬라이드 211–356 인용 (RNN→Transformer→GPT/BERT/RLHF).
- 일반 LLM 원리: Vaswani 2017 (Transformer); Radford 2018/2019 (GPT); Devlin 2019 (BERT); Raffel 2020 (T5); Ba 2016 (LayerNorm).
- 학습·스케일·정렬: Kaplan 2020·Hoffmann 2022 (scaling); Brown 2020 (GPT-3); Ouyang 2022 (RLHF); Rafailov 2023 (DPO).
- 단백질/오믹스 2026: Lin 2023 (ESM-2/ESMFold); Hayes 2024 (ESM3); ESM Cambrian 2024–25; Madani 2023 (ProGen); Wang 2024 (TxGemma); Cui 2023 (scGPT).
- 화학 LM 2026 (3세대): Edwards 2022 (MolT5, arXiv:2204.11817); Chithrananda 2020 (ChemBERTa); ChemDFM-v1.5-8B (Cell RPS 2025, arXiv:2401.14818); ChemLLM-7B-Chat (arXiv:2402.06852); ChemVLM-8B (arXiv:2408.07246); Bagal 2022 (MolGPT).
- 프론티어 지형 2026: GPT-5 계열·Claude 5(Opus/Sonnet)·Gemini 3·Llama 4 — 세부 버전 2026.9 확인 권장 (0905_agent/2026_landscape.md).
- * 원리 그림은 박혜진 교안 도식(CS231n/논문 기반). 최신·2차소스 수치는 확인 권장. 데이터·수치 무·날조 원칙.